



Escuela Politécnica

UNIVERSIDAD DE EXTREMADURA

ESCUELA POLITÉCNICA

GRADO EN INGENIERÍA INFORMÁTICA EN
INGENIERÍA DE COMPUTADORES

TRABAJO FIN DE GRADO

**Estudio de distintos modelos de inteligencia
artificial para la detección de TEA a partir
de mensajes en redes sociales**

Miguel Iglesias Estévez

Junio/Julio, 2026



UNIVERSIDAD DE EXTREMADURA

ESCUELA POLITÉCNICA

GRADO EN INGENIERÍA INFORMÁTICA EN
INGENIERÍA DE COMPUTADORES

TRABAJO FIN DE GRADO

**Estudio de distintos modelos de inteligencia
artificial para la detección de TEA a partir
de mensajes en redes sociales**

Autor: Miguel Iglesias Estévez

Tutor: Miguel Ángel Vega Rodríguez

Resumen

El TEA (Trastorno del Espectro Autista) es una condición del neurodesarrollo caracterizada por desafíos en la interacción social y la comunicación.

Plataformas en línea como Twitter, donde usuarios comparten sus vivencias, se han convertido en recipientes de grandes cantidades de datos sobre las personas, incluidas las que padecen TEA.

En la actualidad, uno de los mayores retos del TEA radica en la dificultad y demora de los procesos de diagnóstico, así que estos datos tienen una gran importancia, ya que pueden ayudar a combatir estos retos si se utilizan para identificar señales de alerta y reducir los tiempos de detección del TEA.

No obstante, el volumen de datos presente en estos medios es tal que resultaría inabarcable analizarlos a mano, por lo que surge la necesidad de procesar la información de forma automática con el objetivo de agilizar los diagnósticos.

Para esto, en este trabajo se han propuesto diversos modelos de inteligencia artificial que ayudan a solucionar este problema: analizan de forma automática estos datos, mensajes de Twitter, para identificar si la persona en cuestión padece TEA o no.

Los distintos modelos han sido evaluados según su capacidad de detección (clasificación) así como el tiempo que requieren, realizándose comparaciones con modelos publicados en la literatura científica.

Palabras clave: Trastorno del Espectro Autista (TEA), Twitter, Inteligencia Artificial.

Abstract

ASD (Autism Spectrum Disorder) is a neurodevelopmental condition characterized by challenges in social interaction and communication.

Online platforms such as Twitter, where users share their life experiences, have become receptacles of large amounts of data on people, including those with ASD.

Currently, one of the greatest challenges of ASD lies in the difficulty and delay of diagnostic processes, so this data is of great importance, as it can help combat these challenges if used to identify warning signs and reduce ASD detection times.

However, the volume of data present in these media is such that analyzing them by hand would be unmanageable, which is why the need arises to process the information automatically with the aim of expediting diagnostics.

For this, in this work, various artificial intelligence models have been proposed that help solve this problem: they automatically analyze this data, Twitter messages, to identify whether that person suffers from ASD or not.

The different models have been evaluated according to their detection (classification) ability as well as the time they require, making comparisons with models published in the scientific literature.

Keywords: Autism Spectrum Disorder (ASD), Twitter, Artificial Intelligence.

Índice general

1. Introducción	1
2. Objetivos	3
3. Antecedentes	4
4. Material y método	7
4.1. Material	7
4.1.1. Hardware	7
4.1.2. Software	8
4.1.3. Datos utilizados	8
4.2. Enfoque de Aprendizaje Automático	9
4.2.1. Preprocesamiento y representación numérica del dataset	10
4.2.2. Logistic Regression	13
4.2.3. Stochastic Gradient Descent (SGDClassifier)	14
4.2.4. Support Vector Classification (SVC)	15
4.2.5. Linear Support Vector Classification (LinearSVC)	16
4.2.6. Multi-Layer Perceptron Classifier (MLPClassifier)	17
4.2.7. Random Forest (RandomForestClassifier)	19
4.3. Paradigma de Aprendizaje Profundo (<i>Deep Learning</i>)	20
4.3.1. Tokenización del dataset	20
4.3.2. Uso de Embeddings Preentrenados GloVe	22
4.3.3. GRU (<i>Gated Recurrent Unit</i>)	23

4.3.4.	CNN (<i>Convolutional Neural Network</i>)	25
4.3.5.	Transformer	27
4.3.6.	Configuración de los entrenamientos	29
4.4.	Modelos de Aprendizaje Preentrenados	30
4.4.1.	DistilBERT	30
4.4.2.	DeBERTaV3: Variantes XS y S	31
4.4.3.	FNet (<i>Fourier-Net</i>)	33
4.4.4.	BERT: Variantes Tiny, Tiny-SST2, Small y Medium	34
4.4.5.	RoBERTa	35
4.4.6.	Estudio de longitudes producidas por los tokenizadores	36
4.4.7.	Configuración de los entrenamientos	37
5.	Resultados y discusión	39
5.1.	Resultados de los modelos de Aprendizaje Automático	39
5.2.	Resultados de los modelos de Aprendizaje Profundo	43
5.3.	Resultados de los modelos de aprendizaje preentrenados	44
6.	Conclusiones	46
	Bibliografía	48

Índice de tablas

4.1. Hiperparámetros seleccionados para las arquitecturas de aprendizaje profundo	29
4.2. Comparativa de tamaño de las variantes DeBERTaV3 frente a BERT-Base	32
4.3. Comparativa de tamaño de las variantes compactas frente a BERT-Base .	34
4.4. Comparativa de percentiles de longitud tras la tokenización del dataset . .	37
4.5. Hiperparámetros empleados por modelo	38
5.1. Resultados de los modelos ML con vectorización <i>Count Vectorizer</i>	40
5.2. Resultados de los modelos ML con vectorización <i>TF-IDF</i>	41
5.3. Resultados de los modelos ML con vectorización mediante <i>Embeddings</i> . .	42
5.4. Resultados de los modelos de aprendizaje profundo	43
5.5. Resultados de los modelos de lenguaje preentrenados	44

Índice de figuras

4.1. Funcionamiento de Count Vectorizer	11
4.2. Resultado de aplicar el Kernel RBF sobre datos no separables linealmente .	16
4.3. Funcionamiento de LinearSVC	17
4.4. Esquema del algoritmo de retropropagación	18
4.5. Funcionamiento de Random Forest	19
4.6. Histograma de las longitudes de los tweets en tokens	21
4.7. Estructura de la arquitectura GRU	24
4.8. Diagrama de bloques del modelo GRU propuesto	24
4.9. Capas habituales de una CNN	25
4.10. Diagrama de bloques de la arquitectura de la CNN multirrama propuesta .	26
4.11. Arquitectura habitual del Transformer	27
4.12. Diagrama de bloques de la arquitectura basada en Transformer propuesta .	28

Capítulo 1

Introducción

El trastorno del espectro autista (TEA) es una condición que afecta al desarrollo cerebral, más específicamente a los patrones de comunicación y a las habilidades sociales de las personas que lo padecen.

Se suelen manifestar los primeros síntomas entre los primeros 18 y 24 meses de edad, así que la prontitud del diagnóstico es un reto a nivel global. A pesar de esto, en España la edad promedio de diagnóstico supera los 5 años [1], lo que dificulta el desarrollo de estas personas y empeora su bienestar emocional.

Adelantar estos diagnósticos ayuda a que estas personas dispongan del apoyo suficiente como para adaptarse de forma correcta y afrontar su vida cotidiana adecuadamente; además de que puede suponer un ahorro de costes sanitarios al reducir la necesidad de atenciones más especializadas y complejas en etapas posteriores de la vida de los pacientes.

Dado que muchas veces los síntomas están presentes en la comunicación, plataformas en línea como Twitter han adquirido bastante relevancia, ya que muchos usuarios las utilizan para expresar sus experiencias de forma espontánea y auténtica, lo que las convierte en fuentes considerables de datos que pueden resultar útiles para la detección de patrones relacionados con condiciones de este tipo.

Sin embargo, la cantidad de datos presentes en estas plataformas es demasiado abundante como para analizarlos de forma manual, de modo que surge la necesidad de implementar herramientas que permitan analizar de forma automática y eficiente estos

datos para extraer patrones que puedan ayudar a la detección del TEA.

Para ello, es especialmente conveniente el uso de modelos de inteligencia artificial (IA). Esta tecnología ha tenido una evolución muy prometedora durante los últimos años, especialmente en el área del Procesamiento del Lenguaje Natural, que es justamente uno de los ámbitos de este trabajo.

Específicamente, hay que realizar una tarea de clasificación binaria, es decir, tomando un tweet como entrada, lo que se pretende hacer es asignar a ese tweet una etiqueta que indique si la persona que lo escribió padece TEA o no.

Este problema ya fue estudiado en [2], de modo que este trabajo toma al artículo citado como punto de partida para proponer modelos adicionales encargados de llevar a cabo esta clasificación, con el objetivo de mejorar los resultados conseguidos por los investigadores en el estudio anterior.

A continuación, se expone la estructura general que va a seguir el documento. Primeramente, tras esta introducción se seguirá por establecer los objetivos, generales y específicos, en el Capítulo 2. Tras eso, el Capítulo 3 abordará los antecedentes existentes previos a este trabajo. Seguidamente, se llevará a cabo en el Capítulo 4 una detallada explicación de los materiales y métodos empleados para la realización de la presente investigación. Finalmente, el Capítulo 5 contendrá y analizará los resultados obtenidos en las diversas pruebas realizadas, para luego finalizar en el Capítulo 6 con las conclusiones extraídas y reflexiones sobre posibles trabajos futuros.

Capítulo 2

Objetivos

El objetivo general de este proyecto es proponer y presentar diversos modelos de inteligencia artificial que sirvan para detectar TEA a partir de mensajes de texto informales escritos en redes sociales, con el fin de obtener en el proceso, a ser posible, resultados mejores que los existentes en la literatura científica.

Más concretamente, los objetivos específicos son:

- Adquirir mayores conocimientos acerca de los métodos de preprocesamiento y arquitecturas empleadas habitualmente en problemas de este tipo, lo que servirá como formación previa a la realización del trabajo.
- Proponer y evaluar modelos de *Machine Learning* (ML) para resolver este problema de clasificación.
- Diseñar arquitecturas de *Deep Learning* basadas en redes neuronales adaptadas a este problema de categorización de textos.
- Ajustar modelos preentrenados avanzados de Procesamiento del Lenguaje Natural (*NLP*, *Natural Language Processing*) con los datos del dataset disponible.
- Evaluar y comparar los resultados obtenidos por los modelos de cada categoría con los propuestos en el estudio previo [2] de la literatura científica.

Capítulo 3

Antecedentes

El único estudio conocido que hace la misma aproximación del problema es el citado anteriormente [2], y, como se ha explicado, el presente proyecto toma al artículo citado como punto de base y referencia.

Se hace necesario primero dar una definición de lo que es el TEA antes de abordar las posibles técnicas que se pueden usar para detectarlo en mensajes en redes sociales. El trastorno del espectro autista (TEA) concentra varias condiciones destacadas por barreras y desafíos en la socialización y en la forma de comunicarse [3].

Para afrontar la detección de dicho trastorno, los autores de referencia usan modelos pertenecientes a tres paradigmas diferentes: el aprendizaje automático (*Machine Learning*), el aprendizaje profundo (*Deep Learning*), y, por último, técnicas pertenecientes al Procesamiento del Lenguaje Natural.

El primer enfoque, de aprendizaje automático, hace referencia a un área de la Inteligencia Artificial que engloba a las técnicas utilizadas con el propósito de que las máquinas resuelvan problemas aprendiendo patrones extraídos a partir de un conjunto de datos dado, sin que estas estén programadas con reglas específicas [4]. Los modelos de aprendizaje automático propuestos por los autores de referencia se detallan a continuación.

El Árbol de Decisión (*Decision Tree*), que se basa en dividir el conjunto de datos en grupos cada vez más uniformes según qué características cumplen esos datos, formando una estructura de árbol. La división continúa hasta cumplir un criterio de parada, como

profundidad máxima del árbol. Cada hoja del árbol es una categoría [5].

Como ampliación del anterior, proponen también el modelo XGBoost (*eXtreme Gradient Boosting*), el cual es un algoritmo de aprendizaje por acoplamiento (*ensemble*), de modelos más simples (los cuales suelen ser árboles de decisión). Cada submodelo nuevo intenta corregir los fallos de los anteriores, hasta conseguir minimizar la pérdida mediante el descenso del gradiente. Se diferencia del *Gradient Boosting* clásico en que emplea regularización (L1/L2) para prevenir el sobreajuste [6].

Usan también el modelo KNN (*K-Nearest Neighbors*), que es comúnmente empleado para clasificación multiclase, pero que también puede usarse para clasificación binaria. Consiste en cargar todos los datos en memoria simultáneamente, para así decidir a qué clase pertenece un dato nuevo en función de la clase mayoritaria a la que pertenecen sus k vecinos más cercanos (usando diferentes métricas de cercanía en función del problema) [7].

Por otro lado, dada la dificultad inherente de hacer frente al presente problema únicamente con modelos de aprendizaje automático, se hace necesario introducir el paradigma del aprendizaje profundo. Este corresponde a un subconjunto de técnicas de inteligencia artificial que se caracterizan por el uso de Redes Neuronales Artificiales con varias capas ocultas. Dichas redes buscan emular la conexión de las neuronas en el cerebro humano, de ahí su nombre [8]. Dentro de este paradigma, los autores de referencia propusieron varias arquitecturas.

En primer lugar, las RNN (*Recurrent Neural Network*). Estas son un subtipo dentro de las redes neuronales artificiales, especializado en el procesamiento de datos secuenciales. Incorporan el mecanismo de recurrencia al usar la salida en el instante t como entrada en el instante $t + 1$ junto al nuevo dato. Así, consiguen algo similar a una memoria a corto plazo de lo que acaban de procesar. Su principal problema es que esta memoria no aguanta lo suficiente, lo que se conoce como problema del *vanishing gradient* [9] [10].

Para resolver ese problema intrínseco a las RNN, introdujeron la arquitectura LSTM (*Long Short-Term Memory*). Se trata de una modificación de la red neuronal recurrente que fue creada específicamente para combatir el problema del *vanishing gradient*. Para ello, introduce una celda de memoria y tres puertas, que se encargan de decidir qué parte

de la información abandona o entra en la memoria y qué parte de la misma se usa como entrada en el siguiente instante. De ese modo la red es capaz de mantener una memoria a largo plazo de información relevante de forma más eficaz que la RNN [10].

Intentaron también mejorar los resultados de LSTM probando su versión bidireccional, Bi-LSTM (*Bidirectional LSTM*), que consiste en el uso de dos LSTM que recorren la frase en sentidos opuestos, de modo que el modelo es capaz de, en teoría, acordarse del contexto que rodea a una palabra dentro de una frase, tanto antes como después de la misma [11].

A pesar de la utilidad de los modelos basados en recurrencias, el análisis de lenguaje informal, y, a menudo, poco contextualizado, requiere el uso de herramientas más potentes. Esto conduce al tercer y último enfoque adoptado en el estudio de referencia: el Procesamiento del Lenguaje Natural (NLP). Se trata de un campo multidisciplinar de la Inteligencia Artificial cuya finalidad principal es otorgarle a las máquinas la habilidad de analizar, comprender, interpretar y generar lenguaje humano de manera eficaz y correcta [12]. Pertenecientes a este campo, se propusieron en el trabajo de referencia dos modelos.

Primeramente, BERT (*Bidirectional Encoder Representations from Transformers*). Este es un modelo de lenguaje preentrenado por Google, que se basa en el componente Encoder [13] de los Transformers. Como su nombre indica, realiza un procesamiento bidireccional del texto, captando todo el contexto alrededor de las palabras. Su ventaja es que, tras ser entrenado de forma inicial, puede ser reentrenado para tareas más específicas requiriendo no demasiados datos, lo que lo hace muy versátil [14].

Adicionalmente, para adoptar un enfoque más especializado en las redes sociales (en Twitter, específicamente), los autores del artículo mencionado plantearon el modelo BERTweet. Se trata de un modelo tipo BERT pero reentrenado en exclusiva con datos provenientes de Twitter, específicamente unos 850 millones de tweets. Esto lo convierte en un modelo ideal para resolver este problema, ya que extrajeron todo su dataset de dicha red social [15].

Capítulo 4

Material y método

En este capítulo se exponen las condiciones en las que se realizaron los experimentos, así como los recursos y herramientas empleados para los mismos. Asimismo, se justificarán también las elecciones de los métodos escogidos para solucionar el problema en cuestión, y en general también cualquier decisión relacionada con su resolución.

4.1. Material

En esta sección se describen todos los recursos que han sido empleados para la realización de las pruebas. Esto incluye recursos *hardware*, *software*, y, por supuesto, el conjunto de datos utilizado para los entrenamientos de los modelos escogidos.

4.1.1. Hardware

A continuación, se muestran las especificaciones del *hardware* empleado para la realización del proyecto:

- **Equipo:** Acer Nitro AN515-45.
- **Procesador (CPU):** AMD Ryzen 7 5800H (8 núcleos).
- **Memoria RAM:** 16 GB SODIMM DDR4.



- **Tarjeta Gráfica (GPU):** NVIDIA GeForce RTX 3060 Mobile (6 GB VRAM).
- **Almacenamiento:** SSD NVMe WDC de 1024 GB.

4.1.2. Software

Seguidamente, se exponen las características y detalles del *software* usado para la programación de los modelos:

- **Sistema operativo:** Ubuntu Linux v22.04.4 LTS.
- **Entorno de desarrollo integrado (IDE):** Visual Studio Code v1.110.1.
- **Lenguaje de programación:** Python v3.12.3.
- **Bibliotecas de preprocesamiento de datos:** Scikit-Learn v1.7.2, NLTK v3.9.2, y Gensim v4.4.0.
- **Bibliotecas de aprendizaje automático:** Scikit-Learn v1.7.2.
- **Bibliotecas de aprendizaje profundo:** TensorFlow v2.20.0, usando su API de Keras de alto nivel.
- **Bibliotecas de procesamiento de lenguaje natural:** Hugging Face Transformers v5.0.0 y KerasHub v0.27.1.

4.1.3. Datos utilizados

Una vez se ha descrito todo el contexto tecnológico que se ha utilizado, el siguiente componente de importancia para la elaboración del trabajo es el conjunto de datos empleado.

Se ha empleado el mismo dataset que en el artículo [2], con el objetivo de replicar también sus experimentos y desarrollar los propios de este trabajo en igualdad de condiciones.



Para construirlo, los autores empezaron por recopilar manualmente perfiles de usuarios que declaraban en sus biografías padecer trastornos TEA, asegurándose de que pertenecen a individuos, y no a organizaciones (como asociaciones de ayuda o entidades de investigación del trastorno).

Tras esa recopilación de los perfiles, emplearon un script de Python encargado de recorrerlos recogiendo aquellos tweets escritos entre el 1 de enero de 2017 y el 31 de enero de 2022, y de etiquetar automáticamente cada uno de ellos según la presencia de alguna de las palabras clave en la biografía del perfil (ya revisado) del autor de dicho tweet.

Los tweets fueron etiquetados con ‘1’, en caso de que el autor padeciese TEA, y con ‘0’ en caso contrario.

Tras todo este proceso, contaban con 1.014.723 tweets recopilados de 252 perfiles diferentes. Una vez conseguidos, los almacenaron en ficheros CSV para su posterior procesamiento. Específicamente, el proceso de limpieza de los datos consistió en:

- Eliminar del dataset todos los retweets y publicaciones duplicadas.
- Suprimir todos los tweets mal formados.
- Borrar todos los tweets que fuesen publicaciones compartidas provenientes de otras redes sociales, como Youtube o Facebook.

Tras esta limpieza inicial, pasaron a tener 404.627 tweets, de los cuales emplearon un subconjunto perfectamente balanceado de 90.000 para el entrenamiento de sus modelos.

Para la elaboración del presente trabajo se ha empleado también un subconjunto balanceado de 90.000 tweets sacados de ese conjunto limpio de 404.627 tweets.

4.2. Enfoque de Aprendizaje Automático

Antes de empezar con la resolución del problema, es fundamental definir el flujo de trabajo que se ha seguido para la realización de los distintos experimentos, para así poder garantizar una estructura ordenada y replicable.



En primer lugar, se realiza la lectura del dataset de disco y su instanciación en memoria, para así poder empezar a manipularlo. Una vez hecho esto, el primer cambio a realizar es la división del dataset en dos conjuntos: uno de entrenamiento y otro de validación.

Con ánimo de mantener la regularidad con respecto al estudio previo, se ha usado exactamente la misma proporción para esta división: el 75 % de los datos se usarán como datos de entrenamiento, y el 25 % restante como conjunto de validación.

A continuación, se debe realizar una transformación del dataset que lo haga apto para ser manipulado por los modelos. Esto es así porque, al ser texto, los datos vienen codificados como cadenas de caracteres (*string*), lo que no constituye un formato válido que pueda ser procesado por un modelo, ya que estos requieren que la entrada sea numérica para poder procesarla mediante cálculo de gradientes, operaciones de álgebra lineal o las transformaciones matemáticas particulares de cada arquitectura.

Las técnicas empleadas en cada caso para transformar las cadenas de caracteres en números serán detalladas en la sección que corresponda, puesto que varían en función del enfoque escogido.

Tras esto, una vez se tiene el dataset correctamente cargado, dividido y representado numéricamente, el siguiente paso es escoger, diseñar y entrenar los modelos encargados de realizar la clasificación.

4.2.1. Preprocesamiento y representación numérica del dataset

A continuación, se detallan los métodos escogidos para obtener la representación numérica del dataset, usados para los modelos del actual enfoque (aprendizaje automático), así como técnicas de preprocesamiento adicionales que se han empleado.

Inicialmente, se consideraron tres métodos:

- **Term Frequency, Inverse Document Frequency (TF-IDF):** Consiste en pasar las palabras a números asignándole a cada una de ellas una puntuación calculada en base a su número de apariciones. Dicha puntuación es calculada siguiendo la Ecuación 4.1. El resultado de esta técnica es una matriz de $N \times M$, donde N es el número total de tweets y M es el número total de palabras del vocabulario,

donde cada celda es la puntuación $TF-IDF$ para esa palabra en ese tweet concreto (puesto que el factor IDF de la puntuación es global para cada palabra, pero el TF cambia en cada tweet). La ventaja es que se le da a cada palabra la importancia que merece, teniendo en cuenta cómo de común es (mediante el factor IDF) y su peso relativo dentro de cada tweet (representado con el factor TF).

$$TF-IDF(\text{palabra}, \text{tweet}, \text{corpus}) = TF(\text{palabra}, \text{tweet}) \times IDF(\text{palabra}, \text{corpus}) \quad (4.1)$$

$$\begin{cases} TF(\text{palabra}, \text{tweet}) = \frac{\text{N.º apariciones de la palabra en el tweet}}{\text{Longitud total del tweet (en palabras)}} \\ IDF(\text{palabra}, \text{corpus}) = \log \left(\frac{\text{N.º total de tweets en el corpus}}{1 + \text{N.º de tweets que contienen la palabra}} \right) \end{cases} \quad (4.2)$$

- **Count Vectorizer (CV):** Transforma cada tweet en un vector de frecuencias absoluto, midiendo cuántas veces aparece cada palabra dentro de ese mismo tweet (sin tener en cuenta al resto del dataset). El resultado de esta técnica también es una matriz de $N \times M$, como en el caso de TF-IDF, solo que aquí cada celda representa tan sólo cuántas veces aparece esa palabra en ese tweet. Este proceso se puede observar en la Figura 4.1.

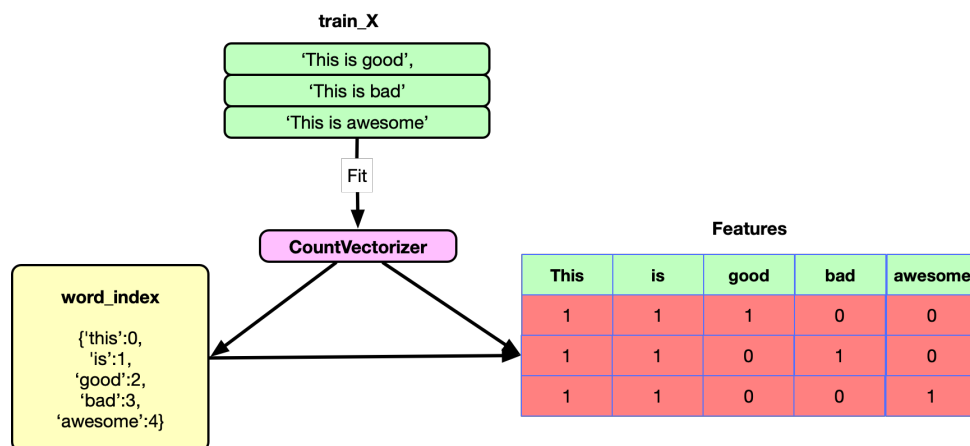


Figura 4.1: Funcionamiento de Count Vectorizer

Fuente: publicado en [Medium](#)



- **Embedding (Word2Vec):** Calcula para cada palabra un vector de características numéricas. Se aplicó en dos fases: primero, se fracciona cada publicación en sus respectivos *tokens*, para luego mapear cada uno de ellos a su respectivo vector de características de tamaño $D = 100$ (valor por defecto de Gensim). Tras esto, con la finalidad de no depender del tamaño variable que puedan tener los tweets sin tener que emplear *padding*, se calcula el vector promedio de todos los *tokens* del tweet, lo que se ve reflejado en la Ecuación 4.3. La salida de este procedimiento aplicado sobre todo el dataset es una matriz de $N \times D$, donde $N = 90,000$ (número de tweets del dataset).

$$\mathbf{V}_{\text{tweet}} = \frac{1}{k} \sum_{i=1}^k \mathbf{v}_i \quad (4.3)$$

Siendo k la cantidad de *tokens* del tweet que están en el vocabulario del *embedding*.

Es necesario usar este método en lugar de emplear padding, ya que, en ese caso, el resultado sería una matriz tridimensional ($N \times L \times D$), siendo L la longitud máxima de los tweets tras aplicar relleno. Dicho formato resultaría problemático ya que todos los modelos de *Machine Learning* de esta etapa admiten únicamente datos bidimensionales, lo que impide proporcionarles matrices de tres dimensiones.

Eso sería todo lo relativo a la representación numérica del conjunto de datos en el paradigma actual. Adicionalmente, como se ha dicho anteriormente, se han implementado técnicas adicionales de preprocesamiento que realizan una limpieza general del conjunto de datos. Estas técnicas adicionales son:

- **Eliminación de las *Stop Words*:** Al tratar con texto dentro del Procesamiento del Lenguaje Natural, es posible encontrar palabras que no aportan información de utilidad al modelo, tales como artículos (“el”, “lo”) o algunas preposiciones (“a”, “con”, “de”). Estas palabras se conocen como *stop words*. En muchos trabajos, es común eliminarlas del conjunto de datos original para conseguir unos datos más depurados de los que los modelos puedan extraer conclusiones más fácilmente.



- **Aplicación de *Porter Stemming*:** El *Porter Stemming* [16] es un algoritmo cuyo propósito es reducir las palabras a su raíz o *stem*. Por ejemplo, tras aplicarlo sobre “pensamiento”, “pensado”, o “pensando”, el resultado sería “pensa”. Se basa en la aplicación de sucesivas reglas que van eliminando progresivamente los sufijos de las palabras hasta obtener su raíz. Su uso puede resultar útil para reducir la dispersión del vocabulario y disminuir el tamaño de la matriz de características resultante, ya sea obtenida mediante *TF-IDF* o *Count Vectorizer*.

Estos métodos, evidentemente, deben aplicarse sobre el conjunto de datos en su formato original, es decir, cuando aún está almacenado como cadenas de caracteres, antes de transformarlo a números.

Como se verá más adelante en el Capítulo 5, el uso de estas técnicas conlleva una ligera mejora de los resultados conseguidos respecto a los entrenamientos en los que no se utilizan.

Todos los experimentos realizados en esta etapa se han hecho utilizando los parámetros por defecto tal y como están establecidos en Scikit-Learn.

Una vez se ha preprocesado y transformado el dataset en estructuras de datos que los modelos puedan procesar, el paso posterior sería la definición de dichos modelos. En el enfoque actual se ha escogido una colección variada de clasificadores supervisados, para tener una variedad lo suficientemente grande como para asegurar no estar omitiendo ninguno potencialmente efectivo para nuestro problema.

A continuación, se describen los modelos estudiados relativos al enfoque de aprendizaje automático.

4.2.2. Logistic Regression

A pesar de llevar el término “regresión” en el nombre, se trata de un método de clasificación que consiste en encontrar la recta (o hiperplano, en caso de que haya muchas variables) que divida mejor (minimizando el error) al conjunto de datos separándolo en las dos clases objetivo: en este caso, que el autor de la publicación padezca TEA o no lo haga. Para esto, este algoritmo toma todas las características de los datos (que previamente han



sido vectorizados usando uno de los métodos expuestos en la subsección anterior) y calcula a partir de ellas una combinación lineal como se puede ver en la Ecuación 4.4, asignando a cada característica un peso β . El resultado de esto es un valor $z \in (-\infty, +\infty)$, que posteriormente es transformado por la función sigmoide, recogida en la Ecuación 4.5, para pasarlo a $(0, 1)$.

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n = \beta_0 + \sum_{i=1}^n \beta_i x_i \quad (4.4)$$

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (4.5)$$

Para medir el error, este algoritmo no usa el error cuadrático medio, típico en las regresiones lineales, sino que emplea la función de pérdida de Entropía Cruzada Binaria (BCE), la cual penaliza mucho al modelo cuando este está muy seguro de un valor, y se equivoca. Esta función de error queda recogida en la Ecuación 4.6.

$$\mathcal{L}_{BCE}(y_i, \hat{y}_i) = -[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (4.6)$$

Donde y_i corresponde a la etiqueta real asociada a ese tweet $[0, 1]$, y $\hat{y}_i$ el valor predicho por el modelo. Como se puede ver, al sustituir y_i por su valor real, uno de los factores de la ecuación se cancela, y el término restante hace que el valor devuelto por la ecuación sea inversamente proporcional a la precisión, generando errores mayores cuanto más alejada esté la predicción de la realidad.

4.2.3. Stochastic Gradient Descent (SGDClassifier)

Además de emplear los optimizadores tradicionales para la actualización de los pesos β , en este proyecto se ha usado también el algoritmo `SGDClassifier` de Scikit-Learn. Tradicionalmente, el algoritmo encargado de modificar los pesos para reducir el error es el Descenso del Gradiente. Sin embargo, este tiene un inconveniente: por cada actualización de los pesos, se calculan los gradientes de todo el dataset entero, lo que puede resultar excesivamente lento si hay muchos datos.



El Descenso del Gradiente Estocástico justamente soluciona este problema: en lugar de calcular el error para todo el dataset, lo hace solo para un tweet (o un número reducido de ellos, lo que se conoce como *mini batch*) seleccionado de forma aleatoria. Esto acelera sustancialmente el tiempo de entrenamiento, y además tiene la ventaja de que al escoger los tweets al azar, aparecen pequeñas variaciones (ruido), lo que ayuda a que el modelo no quede estancado en mínimos locales. El SGD actualiza los pesos siguiendo la Ecuación 4.7.

$$\beta^{(t+1)} = \beta^{(t)} - \alpha \cdot \nabla L_i(\beta^{(t)}) \quad (4.7)$$

Donde $\nabla L_i(\beta^{(t)})$ corresponde al gradiente calculado solo para el tweet i de ese paso, que funciona como un indicador de cuánto y en qué dirección cambia el error ante pequeñas variaciones de los pesos β , los cuales son actualizados en cada etapa (los pesos correspondientes a la siguiente etapa, $\beta^{(t+1)}$, son calculados en base a los de la etapa actual, $\beta^{(t)}$).

Por otro lado, α es lo que se conoce como *learning rate*. Es posible especificarlo como parámetro, pero se ha preferido dejar el valor por defecto usado en el `SGDClassifier` de Scikit-Learn, el cual se calcula de forma dinámica en cada paso según la Ecuación 4.8.

$$\eta^{(t)} = \frac{1}{\alpha \cdot (t + t_0)} \quad (4.8)$$

Donde $\eta^{(t)}$ es el *learning rate* del paso actual t , $\alpha = 0,0001$ y t_0 es un valor inicial establecido por Scikit-Learn para garantizar que la tasa de aprendizaje no sea demasiado pequeña en las primeras etapas. La idea de esto es permitir cambios notables en los pesos al comienzo del entrenamiento con un *learning rate* (LR) grande, e ir disminuyéndolo progresivamente para que el modelo no de saltos bruscos cuando ya está cerca del mínimo.

4.2.4. Support Vector Classification (SVC)

Se dice que los modelos anteriores son lineales porque tratan de encontrar los coeficientes (pesos β) de una combinación lineal de las características. Es decir, intentan encontrar la recta que divide mejor al conjunto de datos en sus categorías correspondientes.

Sin embargo, hay problemas donde no es posible hacer esto porque los datos no son

linealmente separables, esto es, no se les puede separar con un error aceptable usando tan solo una recta. SVC surge para este tipo de problemas: en lugar de intentar buscar una sola recta o plano en las dimensiones originales, lo que hace es aplicar su *kernel* (por defecto RBF o Función de Base Radial), detallado en la Ecuación 4.9.

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2) \quad (4.9)$$

Dicha ecuación calcula la distancia euclídea al cuadrado entre los vectores de características de dos tweets ($\mathbf{x}_i, \mathbf{x}_j$) y la multiplica por el factor $-\gamma$ (un hiperparámetro calculado por Scikit-Learn en base al tamaño del vocabulario y a la varianza de la matriz de características), para luego aplicarle la función $\exp(\cdot)$ y así acotar el resultado a $(0, 1)$.

Esta función proyecta los puntos en un espacio de infinitas dimensiones abstractas, donde ya sí es posible encontrar un hiperplano que divide a los datos con mucho menos error. Cuando se representa este hiperplano en el espacio de características original, se obtiene una frontera curva entre las clases. Este proceso queda recogido en la Figura 4.2.

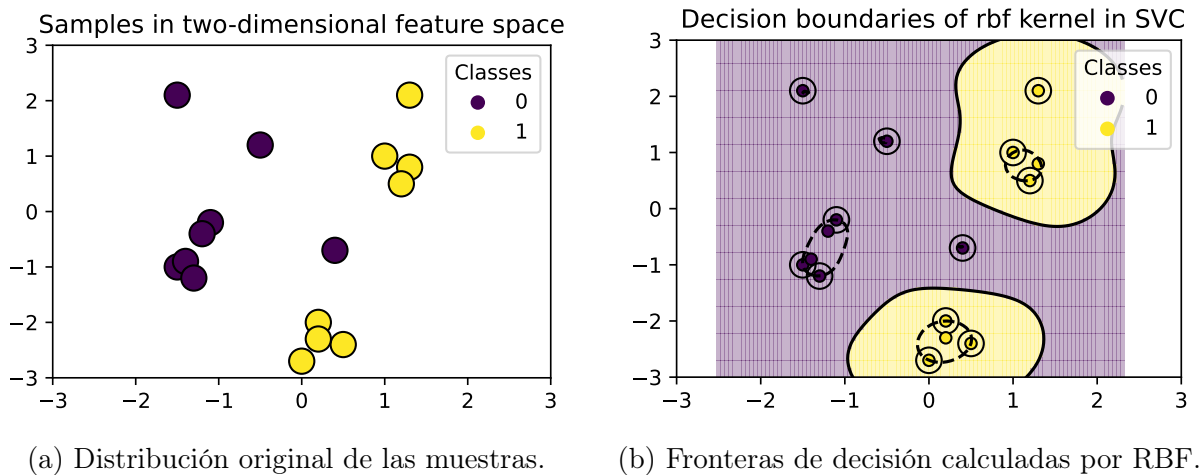


Figura 4.2: Resultado de aplicar el Kernel RBF sobre datos no separables linealmente
Fuente: [Documentación de Scikit-Learn](#)

4.2.5. Linear Support Vector Classification (LinearSVC)

Al contrario que el modelo anterior, el cual produce fronteras de decisión curvas, **LinearSVC** vuelve a ser un clasificador lineal (al igual que **LogisticRegression**), es

decir, asume que los datos son linealmente separables e intenta dividirlos calculando un hiperplano recto. Conceptualmente, es igual que utilizar SVC con un kernel lineal, pero a efectos prácticos difiere bastante de ello, ya que **LinearSVC** se fundamenta por debajo en la librería *liblinear*, optimizada para algoritmos lineales, mientras que SVC utiliza *libsvm*.

Este modelo intenta llegar a la solución maximizando el margen de separación entre las clases, es decir, hallando el hiperplano que maximiza la distancia entre los ejemplos más cercanos de cada clase. Esto es, consigue trazar una frontera entre ambas clases de tal manera que el espacio que queda a ambos lados de la misma es lo más amplio posible. Las muestras que tocan los bordes de este espacio (conocidas como vectores soporte) son las que determinan la posición definitiva de la recta, como se puede ver en la Figura 4.3.

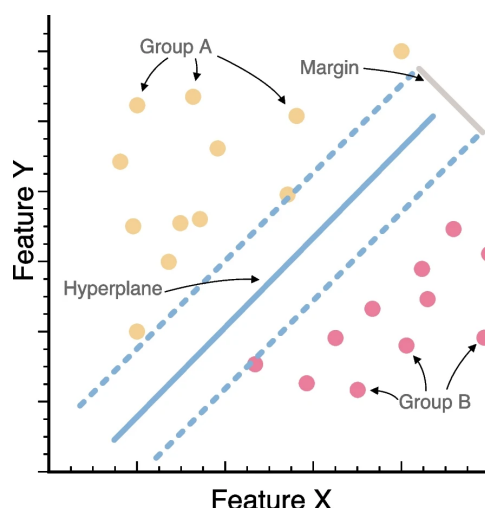


Figura 4.3: Funcionamiento de LinearSVC
Fuente: tomada de [17]

4.2.6. Multi-Layer Perceptron Classifier (MLPClassifier)

A diferencia de los modelos anteriores, que mantienen un enfoque más geométrico, **MLPClassifier** presenta por primera vez en el trabajo las redes neuronales artificiales. Estas no se limitan a hacer transformaciones determinadas a los datos, sino que están diseñadas para detectar patrones ocultos en la información de entrada y están especializadas en muestras con muchas características.

La arquitectura de las redes neuronales artificiales consiste en tres componentes: en

primer lugar, una capa de entrada (*input layer*), en la que cada neurona recibe una sola característica del tweet, a continuación, una o más capas ocultas (*hidden layers*), que procesan el resultado generado por la capa de entrada en busca de información adicional, y por último la capa de salida (*output layer*), que es la que devuelve la predicción sobre si el tweet de entrada pertenece a una clase u otra.

En este proceso, los datos fluyen siempre hacia delante, desde la capa de entrada hasta la de la salida, pasando por todas las capas ocultas: cada neurona calcula su resultado multiplicando la entrada por su peso w , añadiéndoles un sesgo (*bias*), aplicando a eso una función de activación no lineal (para introducir no-linealidad en el modelo, y así hacer que la aplicación de las operaciones de las neuronas no sean equivalentes a una combinación lineal) antes de propagar el resultado hacia la siguiente capa. Todas las neuronas de una capa están conectadas con todas las neuronas de las capas adyacentes (por lo menos en las arquitecturas totalmente conectadas).

El modelo aprende aplicando el algoritmo de retropropagación (*backpropagation*). Tras realizar la predicción, se analiza la diferencia entre el resultado y el valor real (mediante una función de pérdida) y se recorre la red en sentido inverso imputando el error a cada neurona, es decir, modificando su valor w dependiendo de qué impacto haya tenido esa neurona en la predicción final. Este proceso queda recogido en la Figura 4.4.

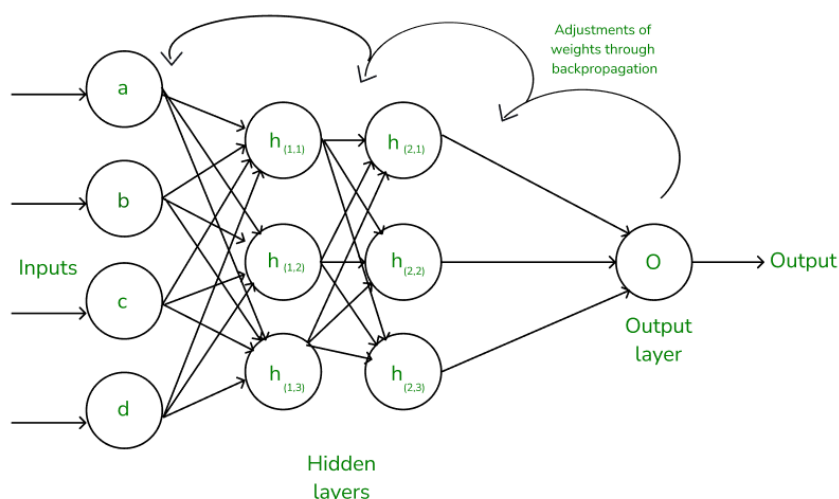


Figura 4.4: Esquema del algoritmo de retropropagación

Fuente: publicado en [MSMK](#)

4.2.7. Random Forest (RandomForestClassifier)

Se dice que todos los modelos expuestos en este apartado son simples porque cada uno de ellos emplea un criterio o algoritmo único internamente para realizar la clasificación. En contraste con esto, puede haber también modelos que aplican el aprendizaje de conjunto (*ensemble learning*). Este es el caso de `RandomForestClassifier`.

En vez de construir un único clasificador por debajo, este modelo crea una estructura formada por múltiples árboles de decisión [5] (de ahí que reciba el nombre de bosque), los cuales son entrenados de forma totalmente independiente.

Para asegurar que haya diversidad entre los árboles (y que así no sean el mismo modelo repetido varias veces) este algoritmo usa varios mecanismos: en primer lugar, cada árbol es entrenado con una muestra distinta del dataset (lo que se conoce como *bootstrap*), la cual es seleccionada de forma aleatoria.

Además, se aplica también un submuestreo de características (*attribute bagging*). Es decir, los árboles no tienen acceso a toda la matriz de características, sino solo a una parte de ella. Estas técnicas fomentan la aparición de disparidad entre los distintos clasificadores.

Usando estos dos criterios se forma un “comité” de modelos variados, que, tras haber sido entrenados, someten a votación el dato de entrada (ya en el proceso de evaluación) para decidir la clase a la que pertenece esa instancia en base al resultado de dicha votación. Este proceso está representado en la Figura 4.5.

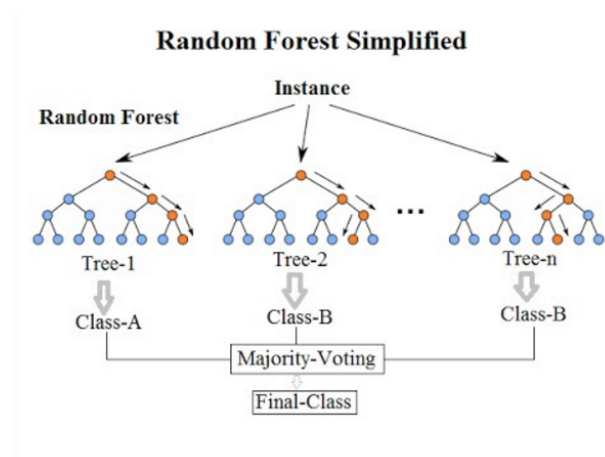


Figura 4.5: Funcionamiento de Random Forest

Fuente: publicado en [Medium](#)



Este proceso de votación queda detallado en la Ecuación 4.10.

$$\hat{Y}_{RF} = \arg \max_{c \in \{0,1\}} \sum_{t=1}^T I(h_t(X) = c) \quad (4.10)$$

Donde T indica el número total de árboles que ha construido el modelo, la función I devuelve 1 si la predicción $h_t(X)$ coincide con el valor de c en ese momento (o 0 en caso contrario), el sumatorio se encarga de acumular cuántas predicciones tiene cada clase $c \in \{0, 1\}$, y la función $\arg \max$ se queda con la clase que ha recibido un número mayor de votos.

4.3. Paradigma de Aprendizaje Profundo (*Deep Learning*)

En este apartado se exponen las arquitecturas de aprendizaje profundo propuestas para la resolución del problema, así como la técnica de representación numérica empleada para los modelos de este paradigma y el uso de un *embedding* preentrenado como base de conocimiento de dichos modelos.

4.3.1. Tokenización del dataset

A diferencia de los modelos de aprendizaje automático, los enfoques planteados en esta sección requieren conservar el orden original en el que los tweets estaban escritos, ya que pueden extraer información a partir del contexto cercano a las palabras.

Por ese motivo, se ha descartado el uso de las formas de representación usadas en la etapa anterior, ya que estas rompen totalmente el orden de los textos al tratarlos como un "saco de palabras" (*bag-of-words*) para formar la matriz de características.

En consecuencia, para la vectorización del dataset en esta etapa, se ha utilizado un enfoque diferente: se ha empleado el **Tokenizer** presente en la librería de TensorFlow, transformando los tweets en secuencias de *tokens* que mantienen el orden inicial.

El proceso, en este caso, es bastante más sencillo que anteriormente: el tokenizador

se encarga de procesar automáticamente los datos de entrenamiento construyendo el vocabulario y estableciendo internamente la correspondencia *token-número*, que luego utiliza para representar las secuencias (tweets) numéricamente.

Tras esto, únicamente hay que agregarle a dichas secuencias un relleno, ya que los modelos requieren una entrada de tamaño regular, cosa que no está garantizada si se mantienen las longitudes originales de los tweets.

A lo largo de este procedimiento, destacan principalmente dos hiperparámetros: la longitud de secuencia establecida y el tamaño máximo del vocabulario.

Los autores del estudio referente emplearon un tamaño máximo del vocabulario de 1000 palabras (de modo que tienen en cuenta tan solo las 1000 palabras más comunes del dataset, el resto se descartan) y una longitud de secuencia de 200 *tokens*, así que los tweets que superan dicha longitud son recortados, y los que no son rellenados con 0.

Sin embargo, se decidió cambiar los valores de dichos hiperparámetros. En primer lugar, se sometió a prueba el dataset disponible con ánimo de estudiar las longitudes de los tweets. Si estos, en promedio, son muy largos, entonces está justificada la elección de una longitud de secuencia razonablemente grande (como 200) para así garantizar que todos caben sin necesidad de ser recortados.

Específicamente, se calculó el histograma representativo de la distribución de dichas longitudes en los tweets del dataset. Dicho histograma puede verse en la Figura 4.6.

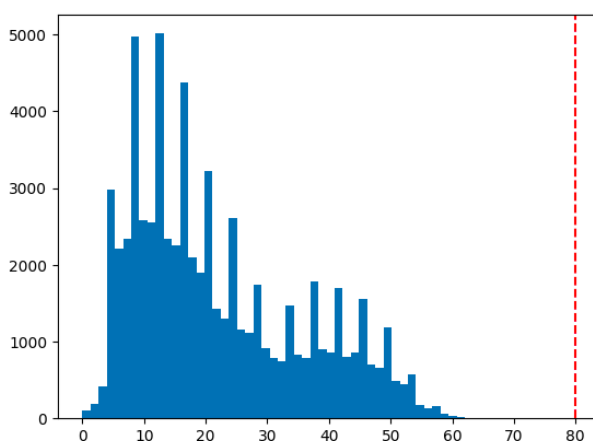


Figura 4.6: Histograma de longitudes de los tweets

Fuente: elaboración propia



Teniendo en cuenta esta distribución, parece que es posible optimizar la longitud de secuencia escogiendo un número más bajo, ya que eso consigue, por un lado, reducir el tiempo de entrenamiento al requerir menos memoria, y, por otro, no introducir demasiado relleno en las secuencias, lo que puede dificultar el aprendizaje de los modelos.

Según el histograma, no hay una cantidad significativa de tweets con una longitud cercana a 60, por lo que se escogió 55 como longitud establecida de secuencia, para asegurar que la mayoría de tweets no necesitan ser recortados y que la cantidad de relleno introducida es mínima.

Por otro lado, se decidió también hacer un aumento del tamaño máximo del vocabulario. Establecerlo tan solo a 1000, como se hace en el trabajo de referencia, tiene el problema potencial de dejar atrás una cantidad significativa de palabras poco comunes pero que pueden aportar información relevante para el problema, sobretodo en un contexto informal como puede ser la red social Twitter, donde es habitual el uso de jerga, abreviaciones, faltas de ortografía...

Por ello, con ánimo de conseguir un vocabulario más amplio y completo, se aumentó el tamaño máximo del mismo a 25000 *tokens*. Aunque pueda parecer un aumento drástico que forzará un uso intensivo de memoria, en realidad este parámetro decide únicamente el tamaño del vocabulario, pero no fuerza a que los modelos tengan que procesar secuencias más grandes durante el entrenamiento, por lo que no tiene un impacto relevante en el tiempo de ejecución de los mismos.

4.3.2. Uso de Embeddings Preentrenados GloVe

Como se verá más adelante, todos los modelos de este paradigma (aprendizaje profundo) cuentan con una capa inicial de *embedding* que se encarga de asignar a cada token de entrada su vector de características correspondiente.

Normalmente, estas características suelen ser parámetros entrenables adicionales del modelo que cambian a lo largo del entrenamiento, al no estar ajustada esta capa de *embedding*.

Sin embargo, es posible sustituir esto por un *embedding* preentrenado que sirva como



base, reduciendo la cantidad de parámetros entrenables y otorgándole al modelo más información acerca de las palabras.

Eso es justamente lo que se ha hecho en este trabajo. Específicamente, se ha decidido utilizar el *embedding* preentrenado GloVe (*Global Vectors for Word Representation*) [18], ya que cuenta con la ventaja de incluir versiones entrenadas con datos sacados de Twitter, lo que se ajusta de forma perfecta al problema presente.

El fichero que contiene las características del *embedding* ha sido descargado de [19], que es el repositorio del grupo de procesamiento de lenguaje natural de la Universidad de Stanford, desde donde se puede acceder a sus modelos.

Concretamente, se ha descargado el directorio que contiene los modelos entrenados con datos de Twitter, del cual se ha usado la versión de 200 características por palabra.

Una vez explicada la técnica de modelado numérico empleada, e introducido el uso del *embedding* de GloVe, se detallan a continuación los modelos propuestos pertenecientes al paradigma de aprendizaje profundo.

4.3.3. GRU (*Gated Recurrent Unit*)

Como se ha explicado anteriormente, las RNN presentan el problema del *vanishing gradient* [9], lo que dificulta que el modelo pueda mantener una memoria a largo plazo de lo que ha procesado hasta el momento. Para solucionar este problema surgieron las redes LSTM [10], vistas anteriormente.

Las redes GRU se idearon como una solución al problema del *vanishing gradient* más eficiente que las LSTM al simplificar la estructura interna. Las principales diferencias entre estas dos arquitecturas se explican a continuación.

Las LSTM gestionan la memoria mediante dos canales diferentes: el *cell state*, encargado de la memoria a largo plazo, y el *hidden state*, que gestiona el corto plazo. Las redes GRU simplifican la gestión de la memoria al controlarla toda (largo y corto plazo) desde una misma unidad (llamada también *hidden state*).

Esta simplificación se observa también en otros aspectos del modelo, como el sistema de control. En lugar de usar las mismas tres puertas que LSTM, las redes GRU gestionan el

flujo de información usando tan solo dos herramientas: la puerta de actualización (*update gate*) y la puerta de reinicio (*reset gate*).

La puerta de actualización de las GRU es una combinación de las puertas de olvido y de entrada de las LSTM, es decir, se encarga de decidir qué parte del conocimiento se mantiene y qué porciones de las nuevas entradas deben añadirse al estado actual. Por otro lado, la puerta de reinicio es la que evalúa qué parte de la memoria ha dejado de ser útil para el contexto actual, lo que permite ir liberando los datos que ya no se necesitan.

La estructura general de la arquitectura GRU se puede ver en la Figura 4.7.

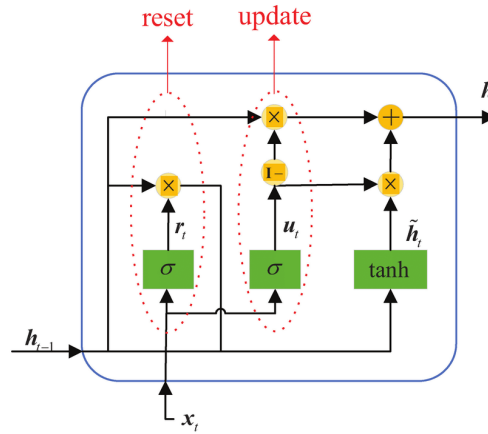


Figura 4.7: Estructura de la arquitectura GRU
Fuente: tomada de [20]

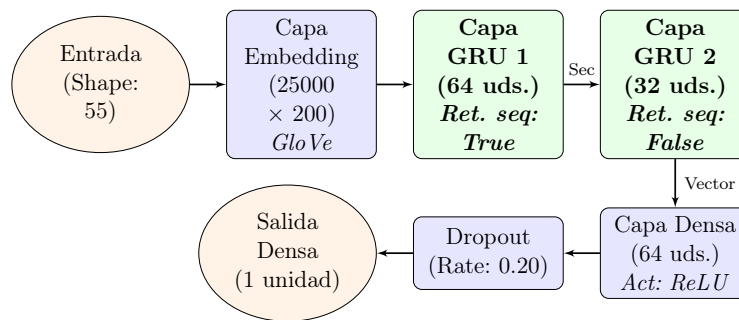


Figura 4.8: Diagrama de bloques del modelo GRU propuesto
Fuente: elaboración propia

Esta simplificación respecto a las LSTM ofrece ventajas relevantes en el entrenamiento, ya que se reduce la cantidad de parámetros entrenables en comparación a un modelo LSTM

similar, lo que se traduce en un menor tiempo de cómputo y menor consumo de memoria.

Específicamente, el modelo GRU propuesto se puede observar en la Figura 4.8.

4.3.4. CNN (*Convolutional Neural Network*)

Las Redes Neuronales Convolucionales (CNN) aprenden aplicando filtros (denominados *kernels*) sobre la entrada para la obtención de patrones locales, como bordes o formas concretas en imágenes, o expresiones concretas dentro de una frase.

A esto se le aplica una capa de agrupación o *pooling*, que resume las características de lo extraído por los filtros anteriores para quedarse únicamente con lo más relevante, que es lo utilizado por las capas finales para hacer la clasificación.

La arquitectura general de una CNN puede verse en la Figura 4.9.

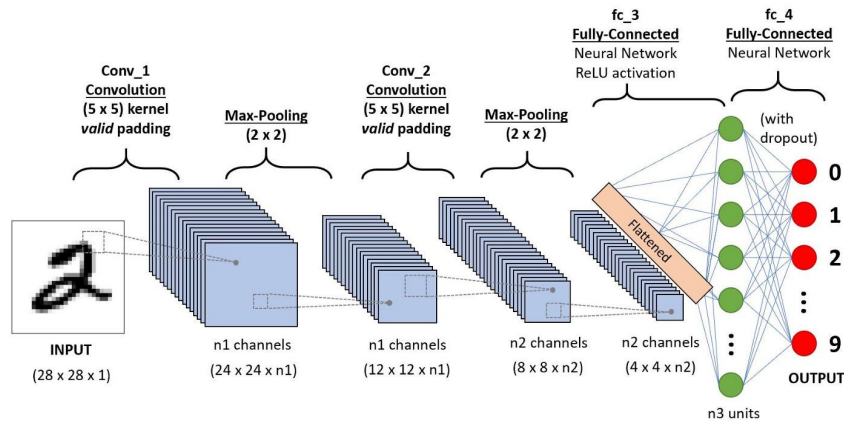


Figura 4.9: Capas habituales de una CNN

Fuente: sacado de [Medium](#)

A pesar de que el uso de las CNN se ha estandarizado dentro del área de procesamiento de imágenes, también es posible su uso en problemas de clasificación de texto, como el presente. Esto es así por varios motivos:

En primer lugar, porque la salida de la capa de *embedding* al pasar una secuencia de texto de tamaño $N = 55$ (que es la longitud establecida de las secuencias, como se comentó anteriormente) es una matriz de $N \times D$, donde D es la dimensión del *embedding*, es decir, el número de características que asocia a cada palabra. En este caso, $D = 200$.

Esto se asemeja notablemente a una imagen en escala de grises, solo que en este caso las filas corresponden a las palabras de la secuencia, y las columnas a sus características.

Esta estructura es procesable por los filtros convolucionales 1D (que son los que se emplean siempre al aplicar CNN a procesamiento de texto), los cuales tienen una dimensión de $k \times D$. De este modo, los filtros pueden desplazarse por el texto procesando todas las características de k palabras consecutivas, o lo que es lo mismo, la evaluación de k -gramas.

La principal ventaja de este planteamiento es la eficiencia. En los modelos basados en recurrencias, el texto se procesa secuencialmente, pero las operaciones de los filtros son fácilmente paralelizables, lo que permite trasladar esa carga de cómputo a la GPU, obteniendo mejores tiempos de entrenamiento, como se verá en el Capítulo 5.

En la Figura 4.10 se presenta el modelo que se planteó para la CNN.

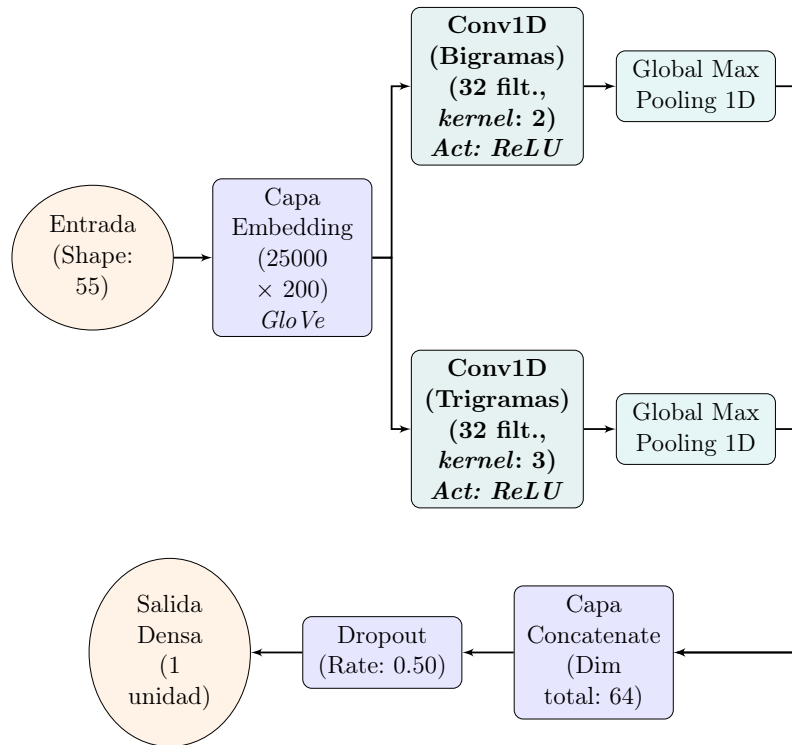


Figura 4.10: Diagrama de bloques de la arquitectura de la CNN multirrama propuesta
Fuente: elaboración propia

Como se puede observar, se han empleado dos filtros convolucionales cuyos resultados son resumidos y luego concatenados en la misma capa de clasificación.

4.3.5. Transformer

A diferencia de GRU, que procesa el texto palabra a palabra, y de la CNN, que analiza pequeñas agrupaciones de palabras en paralelo, el Transformer [21] es capaz de procesar todo el texto simultáneamente mediante el mecanismo de auto-atención (*self-attention*).

Al procesar todo el texto en paralelo, el modelo pierde el orden de la secuencia, lo que se soluciona introduciendo un *positional embedding*. Este se encarga de calcular, para cada palabra, un vector que actúa como firma matemática de la posición de la palabra.

El principal componente del modelo es el mecanismo de auto-atención, que se encarga de medir la relación entre una palabra y el resto de la frase, evaluando todo el contexto que la rodea. Con esto, el modelo es capaz de captar contextos más sutiles (por ejemplo, si al decir "banco", es referido a la entidad financiera o al asiento).

Esto es complementado con un enfoque multi-cabeza (*multi-head attention*), que consiste en lo siguiente: en vez de analizar una sola vez estas afinidades, se analiza la oración desde distintas perspectivas (por ejemplo, se mira la relación entre el sujeto y el verbo, o los nombres que son referidos por cada artículo, etc). Al final, se combinan todas las perspectivas para conseguir una comprensión más profunda del texto.

El modelo tradicional de transformer puede verse en la Figura 4.11.

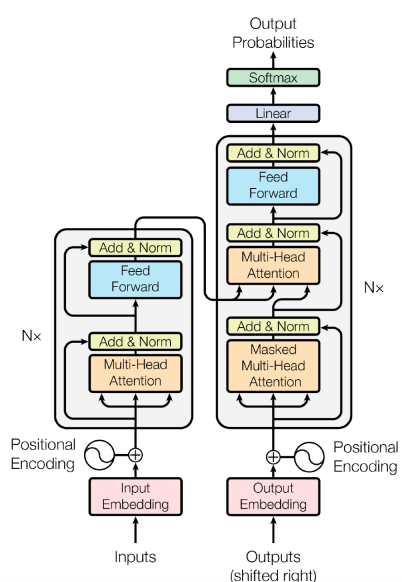


Figura 4.11: Arquitectura habitual del Transformer

Fuente: extraído de [Attention Is All You Need](#)

Por otra parte, la arquitectura específica de Transformer propuesta en el presente trabajo es la expuesta en la Figura 4.12.

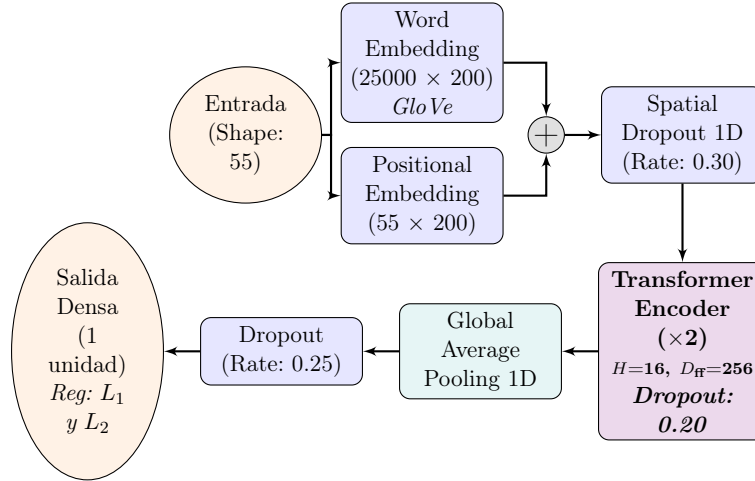


Figura 4.12: Diagrama de bloques de la arquitectura basada en Transformer propuesta
Fuente: elaboración propia

Como se puede observar, se ha prescindido del bloque decodificador (*Decoder*) de la arquitectura Transformer original (situado en su mitad derecha, Figura 4.11). Esto es porque esta unidad es encargada de realizar una predicción de la siguiente palabra en una secuencia de forma autorregresiva, lo que es fundamental para tareas de generación de secuencias de texto de longitud variable. Sin embargo, dado que el problema presente es de clasificación, no es necesario incluirlo en el modelo propuesto.

De ese modo, se han usado en su lugar dos bloques codificadores (*Encoder*), que cuentan con $H = 16$ cabezas de atención y $D_{ff} = 256$ unidades en sus capas densas.

Además, se ha sustituido la capa final de *softmax* (pues esta es la encargada de estimar la probabilidad de la siguiente palabra dentro del vocabulario) por una capa de *Global Average Pooling 1D*, que calcula la media aritmética de las características de todo el tweet, resumiendo todas las propiedades de la secuencia en un único vector, el cual es usado por la capa densa final para calcular la probabilidad de la clasificación.

4.3.6. Configuración de los entrenamientos

A continuación, se especifican los hiperparámetros empleados para los modelos de esta sección. De forma común, en todos se han empleado 14 épocas, y dos conjuntos de datos, de entrenamiento y de validación, que corresponden al 75 % y al 25 % del conjunto original de 90.000 tweets respectivamente.

Los demás hiperparámetros pueden consultarse en la Tabla 4.1.

Modelo	Optimizador	Tasa de Aprendizaje (<i>Learning Rate</i>)	Tamaño de Lote (<i>Batch Size</i>)
GRU (Red Recurrente)	<i>SGD</i>	$8,0 \times 10^{-3}$	64
CNN (Red Convocucional)	<i>Adam</i>	$1,35 \times 10^{-4}$	128
Transformer	<i>Adam</i>	$7,5 \times 10^{-5}$	128

Modelo	Configuración Estructural Avanzada del Optimizador
GRU	• Coeficiente de momento (<i>Momentum</i>): 0.925 • Gradiente acelerado de Nesterov (<i>Nesterov Momentum</i>): True

Tabla 4.1: Hiperparámetros seleccionados para las arquitecturas de aprendizaje profundo
Fuente: elaboración propia

Como se puede observar, CNN y Transformer emplean un mayor tamaño de lote, ya que ambos aprovechan inherentemente mejor el paralelismo de GPU (mientras que GRU, al tener que procesar el texto secuencialmente, no lo hace).

Además, ambos emplean *Adam*, debido a que suele ser el usado por defecto en ambos modelos, aunque no usan el mismo LR debido a que Transformer, al ser más pesado, presenta un mayor riesgo de sobreajuste que hay que prevenir reduciendo su LR.

Por el contrario, GRU emplea SGD con *momentum* y un LR más alto, debido a que el uso de ADAM en redes recurrentes suele ser demasiado agresivo e inestabilizar el entrenamiento. Por ello, se optó por SGD, aumentando notablemente el LR (ya que así lo requiere dicho optimizador) y activando el *momentum* de Nesterov, ya que su omisión provocaba una evolución más inestable.



Cabe destacar, por último, que todos los valores han sido establecidos tras un amplio proceso de experimentación, hasta llegar a lo que se consideran los mejores valores para cada modelo.

4.4. Modelos de Aprendizaje Preentrenados

Se presentan a continuación los modelos preentrenados propuestos en este estudio.

4.4.1. DistilBERT

La creación de modelos de lenguaje preentrenados (como BERT) ha supuesto un aumento de calidad en la resolución de problemas de procesamiento de lenguaje natural. Sin embargo, muchos de estos modelos presentan un inconveniente: cuentan con un alto coste computacional, al tener cientos de millones de parámetros, lo que se traduce en un elevado tiempo de entrenamiento y en la necesidad de usar hardware dedicado, como las GPU, lo que conlleva gastos económicos adicionales.

Por este motivo, han surgido varias arquitecturas alternativas caracterizadas por su optimización y su menor consumo de recursos, lo que las hace idóneas para entornos con infraestructuras limitadas.

Una de ellas es DistilBERT, una versión reducida y ligera de BERT. Se basa en una estrategia denominada destilación de conocimiento (*knowledge distillation*), que consiste en emular una relación profesor-alumno en el entrenamiento, donde cada modelo cumple uno de los roles.

El modelo profesor, correspondiente al BERT original, está ya entrenado, aportando al alumno una base de conocimiento que evita que este tenga que empezar su entrenamiento desde cero.

El alumno, que sería DistilBERT, tiene la mitad de capas que el profesor, y todos sus pesos son inicializados a partir de los de BERT (ya que las capas de ambos modelos coinciden en tamaño). Así, el alumno empieza su entrenamiento con la mitad de conocimientos del profesor. El objetivo del alumno es imitar las predicciones del profesor



y reproducir su comportamiento de la forma más precisa posible.

En este proceso de destilación, el alumno es entrenado siguiendo las probabilidades suavizadas (*soft targets*) establecidas por el profesor. Esto quiere decir que, más que hacer una predicción, lo que se hace es una estimación. Por ejemplo, si se aplica esto a análisis de sentimientos en texto, el modelo podría decir que el texto es humorístico con un 80 % de seguridad e irónico con un 20 % de seguridad. Esta incertumbre, presente en los resultados del profesor, es heredada por el alumno al tomar los pesos del modelo original.

El resultado de este proceso es una arquitectura que, a pesar de reducir el número de parámetros en un 40 % (66 millones frente a los 110 de BERT), conserva el 97 % de las capacidades de análisis de texto del profesor (BERT), consiguiendo unos resultados prácticamente idénticos, lo que la convierte en una alternativa perfectamente válida en caso de querer reducir el tiempo de entrenamiento y el consumo de memoria.

4.4.2. DeBERTaV3: Variantes XS y S

DeBERTaV3 (*Decoding-enhanced BERT with disentangled attention*) es una versión mejorada de BERT caracterizada por la implementación de los siguientes cambios:

- **Mecanismo de atención desacoplada (*disentangled attention*):** Los modelos basados en Transformer (como BERT) consideran la posición de las palabras usando un *positional embedding*, como se comentó en la Subsección 4.3.5, el cual es sumado al vector de características de la palabra. Esto tiene el inconveniente de poder confundir al modelo, ya que puede no saber si dos palabras están relacionadas por su significado o por su posición. Para solucionar esto, DeBERTa separa la posición de las características, guardando ambos datos en vectores independientes. De ese modo, para dos palabras A y B, es posible comparar de forma más robusta las características de la A con las de la B, pero también sus respectivas posiciones con sus contenidos, lo que ofrece al modelo un contexto más amplio.
- **Entrenamiento RTD (*Replaced Token Detection*):** El entrenamiento de BERT consiste en sustituir una de las palabras de la frase por una máscara, para que el

modelo deduzca qué palabra falta. Esto tiene sus limitaciones, ya que la información que extrae BERT de cada frase es reducida. Sin embargo, DeBERTaV3 es entrenado con el método RTD. Esta técnica combina dos modelos: el generador, que cambia una de las palabras por un sinónimo que encaje en el contexto, y el discriminador, que sería DeBERTaV3, que analiza la frase entera y detecta qué palabra ha sido cambiada. Así, el modelo aprovecha más eficientemente los datos, al hacer un análisis detallado de todos los *tokens* para saber cuáles son originales y cuáles no.

- **Compartición de *embeddings* sin conflicto de gradientes:** El discriminador y el generador tienen cometidos incongruentes compartiendo el mismo diccionario: uno quiere juntar las palabras similares en el mismo espacio vectorial para encontrar palabras equivalentes, y el otro quiere separarlas todo lo posible para detectar cualquier mínima anomalía que pueda decirle si es la palabra falsa. La solución a esto, implementada en DeBERTaV3, es delimitar los gradientes de ambos modelos para que puedan cooperar sin anularse mutuamente.

Específicamente, se han empleado las versiones XS (*Extra Small*) y S (*Small*). La comparativa de los tamaños de ambas con BERT-Base queda recogida en la Tabla 4.2.

Parámetro de la arquitectura	DeBERTaV3-XS	DeBERTaV3-S	BERT-Base
Profundidad (Capas L)	12	6	12
Dimensión Oculta (d_{model})	384	768	768
Cabezas de Atención (H)	6	12	12
Dimensión Intermedia (D_{ff})	1536	3072	3072
Tamaño de Vocabulario	128.000	128.000	30.522
Parámetros del Núcleo (<i>Backbone</i>)	22 M	44 M	85 M
Parámetros de Entrada (<i>Embeddings</i>)	48 M	98 M	25 M
Parámetros Totales	70 M	142 M	110 M
% Tamaño del Núcleo (vs. BERT)	$\approx 25,88 \%$	$\approx 51,76 \%$	100,00 %

Tabla 4.2: Comparativa de tamaño de las variantes DeBERTaV3 frente a BERT-Base
Fuente: elaboración propia



4.4.3. FNet (*Fourier-Net*)

FNet es un modelo presentado por investigadores de Google Research en [22], el cual tiene un objetivo similar al de DistilBERT: obtener resultados lo más cercanos a los de BERT pero disminuyendo el tiempo de entrenamiento y el uso de memoria.

Para ello, la estrategia implementada en FNet es prescindir por completo del mecanismo de auto-atención, sustituyéndolo por una transformada de Fourier. El motivo es sencillo: el mecanismo de auto-atención examina qué relación hay entre cada palabra de la frase y todas las demás, lo que tiene una complejidad de $O(n^2)$. Esto significa que si la longitud de la frase a analizar se duplica, el número de comparaciones que tiene que realizar el modelo se cuadruplica, lo que hace aumentar rápidamente los tiempos de entrenamiento.

Para solucionar esto, se emplea la Transformada Discreta de Fourier de dos dimensiones (2D DFT) [23] como operación de mezcla aplicada en dos etapas:

- **Mezcla espacial (*Token mixing*):** Se aplica la Transformada de Fourier Unidimensional (1D DFT) sobre las columnas de la matriz de características analizando cómo varían las propiedades a lo largo de la frase, comparando las palabras según su posición.
- **Mezcla de características (*Hidden mixing*):** Una vez se han mezclado espacialmente los *tokens*, se vuelve a aplicar la 1D DFT, pero en esta ocasión a lo largo de las filas (es decir, las características completas de cada palabra), extrayendo relaciones sutiles entre las propiedades de una misma palabra.

Este enfoque presenta dos ventajas: para empezar, disminuye drásticamente el número de parámetros entrenables del modelo, puesto que la 2D DFT es una operación matemática fija que no requiere ajustar valores a lo largo del entrenamiento (como sí pasa con la auto-atención). La otra ventaja es que se consigue una complejidad de $O(n \log n)$, correspondiente a la implementación del algoritmo de la transformada de Fourier detallado en [24], lo que mejora el orden de complejidad respecto al mecanismo de auto-atención.

4.4.4. BERT: Variantes Tiny, Tiny-SST2, Small y Medium

A diferencia de los apartados anteriores, en los que se han propuesto arquitecturas que introducen cambios conceptuales con respecto al modelo BERT base, ahora no se pretende modificar las cualidades del algoritmo, sino proponer versiones más compactas y ligeras de BERT.

Es decir, a pesar de que este modelo ya fue propuesto en [2], se busca analizar en qué medida es posible reducir el tamaño del modelo sin empeorar significativamente el resto de métricas evaluadas (*accuracy* y F1).

Específicamente, se entrenaron las variantes *Tiny*, *Tiny-SST2* (la cual es del mismo tamaño que *Tiny*, pero fue preentrenada con el dataset SST-2, usado en análisis de sentimientos en texto), *Small*, y *Medium*.

La comparativa de tamaños y número de parámetros entrenables entre todas las versiones y el modelo base se muestra en la Tabla 4.3.

Parámetro de la arquitectura	BERT-Tiny	BERT-Tiny-SST2	BERT-Small	BERT-Medium	BERT-Base
Profundidad (Capas L)	2	2	4	8	12
Dimensión Oculta (d_{model})	128	128	512	512	768
Cabezas de Atención (H)	2	2	8	8	12
Dimensión Intermedia (D_{ff})	512	512	2048	2048	3072
Tamaño de Vocabulario	30.522	30.522	30.522	30.522	30.522
Parámetros del Núcleo (<i>Backbone</i>)	0,5 M	0,5 M	13,5 M	26 M	85 M
Parámetros de Entrada (<i>Embeddings</i>)	3,9 M	3,9 M	15,6 M	15,6 M	25 M
Parámetros Totales	4,4 M	4,4 M	29,1 M	41,6 M	110 M
% Tamaño del Núcleo (vs. BERT)	$\approx 0,58 \%$	$\approx 0,58 \%$	$\approx 15,88 \%$	$\approx 30,58 \%$	100,00 %

Tabla 4.3: Comparativa de tamaño de las variantes compactas frente a BERT-Base
Fuente: elaboración propia

La elección de estas variantes permitirá medir el impacto que tiene la reducción del tamaño de los modelos en su desempeño al realizar la tarea.

Como se puede observar, el tamaño del *Backbone* de BERT-TINY es apenas de un 0,58 % respecto al modelo base. Previsiblemente, esto supondrá una reducción considerable del tiempo de entrenamiento, aunque será necesario debatir si dicha mejora en el tiempo



de ejecución compensará el correspondiente empeoramiento en la exactitud (*accuracy*) y en la métrica F1.

El análisis detallado de los resultados conseguidos por estas variantes, junto a la mencionada reflexión, se realizará en el Capítulo 5.

4.4.5. RoBERTa

RoBERTa (*Robustly Optimized BERT Approach*) es una alternativa optimizada del modelo BERT original, que fue presentada en [25] por investigadores de Meta y de la Universidad de Washington.

En dicho estudio, la hipótesis principal es que el desempeño del modelo BERT original puede ser mejorado aumentando la cantidad de datos y cambiando ciertas decisiones de diseño de BERT, que serían las siguientes:

- **Enmascaramiento dinámico (*Dynamic Masking*):** Como se ha mencionado en la Subsección 4.4.2, en el entrenamiento de BERT se usa la técnica de enmascarar un *token* de la frase para que el modelo deduzca a qué palabra corresponde. En BERT, esto solo se hace una vez antes del entrenamiento, de forma estática, lo que significa que el modelo veía, durante el entrenamiento, siempre la misma frase con la misma palabra tapada. La mejora de RoBERTa consiste en hacer este enmascaramiento de forma dinámica, tapando una palabra distinta en cada época, escogida de forma aleatoria, para que el modelo no pueda memorizar ese contexto.
- **Eliminación de la predicción de la siguiente frase:** Junto al enmascaramiento estático, BERT se entrenaba tratando de determinar si una frase B va o no después de una frase A. En RoBERTa, se ha eliminado este mecanismo, empleando para el entrenamiento textos más largos hasta cubrir los 512 *tokens*, y se descubrió que la predicción de la siguiente frase empeoraba el rendimiento, por lo que al suprimirla se simplificó el proceso e incluso mejoró procesando textos más largos.
- **Aumento de la cantidad de datos:** Mientras que BERT fue entrenado con 16GB de datos y un *batch size* de 256 secuencias, RoBERTa usó un dataset diez veces



más grande (160GB) y aumentó el *batch size* a 8000 frases simultáneas. Esto resulta ventajoso, ya que particularmente los modelos grandes de IA (en los que empezamos a hablar de cientos de millones de parámetros) se benefician de cantidades grandes de datos.

- **Aumento del tamaño del vocabulario:** BERT emplea un vocabulario de 30.522 *tokens*, mientras que RoBERTa eleva dicha cantidad a 50.265. Esto garantiza que el modelo puede reconocer prácticamente cualquier palabra, incluyendo las más raras o las faltas de ortografía, lo que resulta útil para el problema presente.

Por tanto, la propuesta de RoBERTa dentro del presente estudio está orientada a la mejora de los resultados conseguidos en la literatura previa, ya que en el artículo de referencia los autores emplean BERT, específicamente BERT-Base y BERTweet.

El estudio de los resultados conseguidos por este modelo en el Capítulo 5 servirá para determinar si la adopción de las mejoras explicadas es eficaz para superar a los modelos mencionados.

4.4.6. Estudio de longitudes producidas por los tokenizadores

A diferencia de las soluciones propuestas en las secciones anteriores, estos modelos no requieren la implementación de técnicas para la vectorización del texto, puesto que cada uno de ellos incluye por defecto su propio algoritmo de tokenización, que ha sido entrenado conjuntamente con el resto de la arquitectura bajo las mismas condiciones. Esto elimina la necesidad de implementar manualmente técnicas como *TF-IDF* o *Count Vectorizer*, vistas anteriormente.

Sin embargo, antes de comenzar con el entrenamiento, es necesario hacer un estudio previo, similar al que se hizo en la Subsección 4.3.1, con la finalidad de observar las longitudes de las secuencias del dataset producidas por cada tokenizador particular de cada modelo, para así emplear para cada modelo la longitud de secuencia óptima que minimiza la cantidad de relleno necesaria sin cortar demasiados tweets.

Para llevar a cabo el análisis, se emplearon los distintos tokenizadores, calculando



para cada uno distintas métricas (la mediana, el percentil 90, el percentil 95, y el máximo absoluto), como se puede observar en la Tabla 4.4.

Métrica	DistilBERT	Familia BERT	DeBERTaV3	FNet	RoBERTa
Mediana (P_{50})	35	35	35	38	36
Percentil 90 (P_{90})	73	73	72	79	75
Percentil 95 (P_{95})	85	85	84	92	87
Máximo Absoluto	141	141	489	265	380
Múltiplo de 8 óptimo	88	88	88	96	88

Tabla 4.4: Comparativa de percentiles de longitud tras la tokenización del dataset
Fuente: elaboración propia

Como se puede ver, se ha especificado para cada tokenizador el múltiplo de 8 más cercano al percentil 95 en cada caso particular. Este valor se resalta en la tabla porque será precisamente la longitud de secuencia establecida para cada modelo: el 95 % de los tweets tienen una longitud igual o menor, por lo que solo se recorta una minoría y se emplea un *padding* mínimo, pero en múltiplo de 8 para que sea gestionado más eficientemente por la GPU, que habitualmente divide los datos en bloques de tamaños similares.

4.4.7. Configuración de los entrenamientos

En este apartado se exponen los parámetros empleados en los entrenamientos de los modelos introducidos en esta sección.

El conjunto de datos empleado para todos los entrenamientos, de 90.000 tweets, ha sido dividido en dos subconjuntos: uno del 75 % del tamaño original para el entrenamiento y el 25 % restante para la validación, ya que es la misma proporción empleada en el artículo de referencia [2], y han sido organizados en lotes de tamaño 32 para todos los modelos.

Adicionalmente, en todos los entrenamientos se ha empleado también el optimizador AdamW, debido a la posibilidad de introducirle regularización, la cual ha sido establecida en forma de *weight decay*, con un valor de 0.05 para todos los modelos, lo suficientemente grande para penalizar valores grandes de los pesos y así prevenir el sobreajuste.



El resto de parámetros pueden consultarse en la Tabla 4.5.

Modelo	Tasa de Aprendizaje (<i>Learning Rate</i>)	Épocas
DistilBERT	$1,0 \times 10^{-5}$	3
DeBERTaV3 XSmall	$2,3 \times 10^{-5}$	3
DeBERTaV3 Small	$1,0 \times 10^{-5}$	3
FNet	$7,35 \times 10^{-6}$	3
BERT Tiny	$3,65 \times 10^{-5}$	4
BERT Tiny-SST2	$3,65 \times 10^{-5}$	4
BERT Small	$1,5 \times 10^{-5}$	3
BERT Medium	$1,5 \times 10^{-5}$	3
RoBERTa	$2,0 \times 10^{-5}$	3

Tabla 4.5: Hiperparámetros empleados por modelo
Fuente: elaboración propia

De la tabla se pueden extraer varias conclusiones: por un lado, el *learning rate* (LR) ha sido asignado de forma inversamente proporcional al tamaño de cada modelo. Cuanto mayor, menor LR, puesto que los más grandes son los más propensos al sobreajuste, lo que es más fácil que ocurra si hay un LR grande.

Por último, el número de épocas tampoco ha sido definido de forma fija, sino que ha sido establecido por el **EarlyStopping** de Keras, que detiene los entrenamientos en cuanto hay sobreajuste. De ese modo, se ha acabado usando el número de épocas que puede aguantar cada modelo sin ser sobreajustado.

Cabe destacar que los valores exactos de los parámetros no han sido escogidos de forma arbitraria, sino que se ha seguido un riguroso proceso de prueba y error, hasta alcanzar aquellos valores que demostraban una mejor evolución de cada modelo durante el entrenamiento.

Capítulo 5

Resultados y discusión

En la presente sección se presentarán los resultados conseguidos por los distintos modelos propuestos durante los experimentos, haciendo hincapié en la eficacia de los distintos modelos en la resolución del problema (medida en términos de *Accuracy* y *F1-Score*, dos métricas ampliamente conocidas y utilizadas en IA), y en relación al tiempo de entrenamiento requerido por cada uno de ellos.

Tras exponer las métricas conseguidas por los modelos de cada enfoque, se procederá a su comparación con las obtenidas en el estudio referente [2].

5.1. Resultados de los modelos de Aprendizaje Automático

A continuación, se exponen los resultados conseguidos por los modelos expuestos en la Sección 4.2.

Se han creado tres tablas: primeramente, la Tabla 5.1, concentra los resultados conseguidos al representar numéricamente el dataset con *Count Vectorizer*, la Tabla 5.2 recoge los resultados al vectorizar empleando *TF-IDF*, y, por último, la Tabla 5.3 recopila los resultados al emplear *embedding* para transformar el dataset.

En todas ellas se han incluido los resultados conseguidos por los modelos de aprendizaje automático propuestos en el artículo referente, con ánimo de facilitar las comparaciones

de métricas y las discusiones.

Modelo	T (s)	Acc.	F1						
<i>Modelos del artículo de referencia</i>									
Decision Tree	0,649	0,609	0,718						
XGBoost	1,918	0,721	0,740						
KNN	25,046	0,602	0,659						
<i>Modelos propuestos</i>	Count-Vect			+ Stop Words			+ Stemming		
	T (s)	Acc.	F1	T (s)	Acc.	F1	T (s)	Acc.	F1
Logistic Regression	23,416	0,781	0,784	13,325	0,781	0,786	32,057	0,808	0,810
SGDClassifier	0,480	0,781	0,788	0,264	0,778	0,787	0,532	0,802	0,806
SVC	1513,115	0,769	0,782	1169,571	0,775	0,786	1182,107	0,799	0,805
LinearSVC	15,498	0,774	0,773	11,208	0,773	0,772	18,730	0,794	0,795
MLPClassifier	5153,523	0,777	0,776	5282,843	0,766	0,765	5082,142	0,807	0,808
Random Forest	334,285	0,757	0,779	336,084	0,763	0,779	325,025	0,778	0,793

Tabla 5.1: Resultados de los modelos ML con vectorización *Count Vectorizer*
Fuente: elaboración propia

En la Tabla 5.1 hay varios resultados que merece la pena comentar. En primer lugar, hay un gran contraste entre los tiempos de entrenamiento: el más rápido con diferencia, *SGDClassifier*, tarda en entrenarse una cantidad de tiempo mínima en comparación al resto, pero consiguiendo resultados muy similares o incluso mejores.

Además, cabe destacar que usando *Count Vectorizer*, eliminar las *stop words* no parece tener un impacto significativo, mientras que la aplicación de *Porter Stemming* [16] sí que consigue mejorar consistentemente los resultados.

Adicionalmente, en general se observa una mejora considerable de las métricas conseguidas por los modelos propuestos respecto a las obtenidas en el trabajo referente, especialmente en los modelos *Logistic Regression*, *SGDClassifier* y *MLPClassifier*, aunque este último cuenta con un tiempo de entrenamiento excesivo.

A continuación, se exponen los resultados conseguidos al vectorizar el conjunto de datos empleando *TF-IDF* (Tabla 5.2).

Modelo	T (s)	Acc.	F1						
<i>Modelos del artículo de referencia</i>									
Decision Tree	0,649	0,609	0,718						
XGBoost	1,918	0,721	0,740						
KNN	25,046	0,602	0,659						
<i>Modelos propuestos</i>	TF-IDF			+ Stop Words			+ Stemming		
	T (s)	Acc.	F1	T (s)	Acc.	F1	T (s)	Acc.	F1
Logistic Regression	15,275	0,760	0,757	7,447	0,763	0,760	21,468	0,783	0,782
SGD Classifier	0,491	0,757	0,762	0,195	0,758	0,761	0,496	0,783	0,791
SVC	1585,795	0,778	0,783	927,323	0,778	0,784	1746,714	0,804	0,806
LinearSVC	184,167	0,722	0,719	94,911	0,719	0,714	238,262	0,741	0,739
MLP Classifier	1730,510	0,778	0,783	2686,801	0,776	0,776	1707,861	0,799	0,800
Random Forest	331,212	0,761	0,781	331,451	0,766	0,781	328,368	0,778	0,793

Tabla 5.2: Resultados de los modelos ML con vectorización *TF-IDF*

Fuente: elaboración propia

Al emplear *TF-IDF*, se observan patrones similares: de igual modo que en la anterior tabla, se puede ver que eliminar las *stop words* no tiene un impacto significativo, mientras que la aplicación de *Porter Stemming* sí consigue mejorar los resultados de forma general.

Igualmente, *SVC* y *MLPClassifier* siguen siendo los modelos más lentos del conjunto, aunque al usar *TF-IDF* se reducen sus tiempos de entrenamiento, lo que tiene una explicación clara: al tomar en cuenta la frecuencia inversa de los términos en el corpus, muchos de ellos acaban con puntuaciones más cercanas a 0, lo que facilita que los modelos aprendan antes que esos términos tienen menos relevancia (a pesar de ser más comunes), convergiendo antes.

A pesar de que empleando *TF-IDF* también se consigue mejorar los resultados obtenidos por los modelos del artículo referente, la mejora no es tan buena en términos de exactitud y F1 como la conseguida usando *Count Vectorizer*, aunque sí que hay una reducción significativa en los tiempos de entrenamiento de los modelos más lentos, como se ha comentado.

Ahora se exponen los resultados al vectorizar el dataset usando el *embedding word2vec* (Tabla 5.3), que plantea un enfoque diferente al calcular para cada palabra un vector de características, como se ha explicado anteriormente.

Aquí, como se puede observar, lamentablemente los resultados obtenidos no son tan

Modelo	Tiempo (s)	Accuracy	F1-Score
<i>Modelos del artículo de referencia</i>			
Decision Tree	0,649	0,609	0,718
XGBoost	1,918	0,721	0,740
KNN	25,046	0,602	0,659
<i>Modelos propuestos</i>			
Logistic Regression	5,159	0,651	0,672
SGD Classifier	0,712	0,649	0,676
SVC	398,818	0,717	0,732
LinearSVC	6,747	0,652	0,677
MLP Classifier	56,849	0,715	0,717
Random Forest	81,441	0,701	0,710

Tabla 5.3: Resultados de los modelos ML con vectorización mediante *Embeddings*
Fuente: elaboración propia

buenos como los conseguidos al usar *Count Vectorizer* o *TF-IDF*, por lo que claramente no merece la pena el uso de *embedding* al tratar con modelos de aprendizaje automático.

Por eso mismo, en vista de que esta técnica no supone una mejora significativa respecto a las demás, se decidió no ampliar los experimentos de la misma combinándola con la eliminación de las *stop words* o aplicando *Porter Stemming*.

En vista de los resultados expuestos, se pueden extraer varias conclusiones: en primer lugar, es destacable mencionar que la mejor técnica de vectorización, en base a los resultados, es *Count Vectorizer*, especialmente si se combina con *Porter Stemming*. Teniendo en cuenta esto, el mejor modelo de los propuestos es *LogisticRegression*, el cual consigue una exactitud de 0,808 y una puntuación F1 de 0,810, en un tiempo razonable de 32,057 segundos, al ser entrenado con la combinación mencionada.

En caso de que dicho tiempo de entrenamiento resultase un problema, el mejor modelo sería *SGDClassifier* usando la misma combinación, ya que es entrenado en un tiempo de tan solo 0,532 segundos consiguiendo una pérdida mínima en exactitud y en F1.

Estos resultados suponen una mejora clara respecto a los existentes en la literatura científica en *Machine Learning*, conseguidos por los autores de referencia.



5.2. Resultados de los modelos de Aprendizaje Profundo

A continuación, se muestran en la Tabla 5.4 los resultados conseguidos por los modelos de aprendizaje profundo planteados.

Modelo	Tiempo (s)	Accuracy	F1-Score
<i>Modelos del artículo de referencia</i>			
RNN	271,151	0,685	0,711
LSTM	145,924	0,694	0,639
Bi-LSTM	200,360	0,700	0,660
<i>Modelos propuestos</i>			
GRU	120,222	0,793	0,785
CNN	28,072	0,795	0,786
Transformer	173,471	0,802	0,793

Tabla 5.4: Resultados de los modelos de aprendizaje profundo
Fuente: elaboración propia

Hay varios aspectos a destacar: para empezar, tal y como se mencionó en la Subsección 4.3.3, las simplificaciones que introduce la arquitectura GRU respecto a LSTM suponen un aumento del rendimiento y una disminución en los tiempos de entrenamiento, lo que sin duda se ve ratificado en la tabla anterior.

Lo mismo sucede con la CNN: como se explicó en la Subsección 4.3.4, las operaciones de los filtros son fácilmente trasladables a la GPU por su naturaleza paralelizable, lo que permite conseguir un mejor tiempo de entrenamiento.

Por lo demás, los mejores resultados se obtienen empleando el modelo Transformer, ya que, por un lado, se consigue entrenar en menos de 3 minutos (una cantidad de tiempo muy razonable tratándose de aprendizaje profundo) y obtiene unas métricas de exactitud y de F1 de 0,802 y de 0,793, manteniendo la calidad de los resultados conseguidos anteriormente en la Sección 5.1.

En caso de que el tiempo de entrenamiento de 173,471 segundos resultase problemático, el mejor modelo sin lugar a duda sería la CNN, consiguiendo una mejora sustancial del

rendimiento sin sacrificar apenas exactitud ni F1, tal y como se puede ver en la tabla.

Estos resultados siguen suponiendo una mejora significativa respecto a los de la literatura científica en aprendizaje profundo, ya que se ha conseguido una mejora de 10 puntos en promedio consiguiendo mantener o incluso reducir los tiempos de entrenamiento.

5.3. Resultados de los modelos de aprendizaje preentrenados

Seguidamente, se presentan en la Tabla 5.5 los resultados obtenidos tras los entrenamientos de los modelos preentrenados.

Modelo	Tiempo (s)	Accuracy	F1-Score
<i>Modelos del artículo de referencia</i>			
BERT	5093,940	0,849	0,849
BERTweet	4993,970	0,872	0,872
<i>Modelos propuestos</i>			
DistilBERT	672,060	0,833	0,833
DeBERTaV3 (XS)	900,720	0,842	0,842
DeBERTaV3 (S)	1342,890	0,853	0,853
FNet	1023,900	0,824	0,824
BERT (Tiny)	62,330	0,800	0,800
BERT (Tiny-SST2)	59,530	0,800	0,800
BERT (Small)	232,490	0,820	0,820
BERT (Medium)	477,860	0,831	0,831
RoBERTa	1985,500	0,866	0,866

Tabla 5.5: Resultados de los modelos de lenguaje preentrenados
Fuente: elaboración propia

En primer lugar, cabe señalar que en los resultados de los modelos preentrenados hay una perfecta coincidencia entre exactitud y F1, fenómeno que no se da en los resultados de los modelos anteriores.

Además, los valores de exactitud y F1 parten siempre del 80 %, es decir, muy cercanos al mejor valor alcanzable por los modelos de aprendizaje automático y aprendizaje

profundo. Esto es un indicativo de la calidad superior de los modelos de aprendizaje preentrenados (basados en procesamiento del lenguaje natural), capaces de conseguir mejores valores de exactitud y F1.

Es importante también hacer mención a las explicaciones dadas en la Sección 4.4: tal y como se estimaba, las arquitecturas DistilBERT, DeBERTaV3 (en todas sus variantes), FNet, y BERT en sus versiones reducidas, consiguen mejoras significativas en el tiempo de ejecución respecto a los modelos existentes en la literatura, precisamente gracias a las optimizaciones que se plantean en dichas arquitecturas, ya explicadas, que logran disminuir la cantidad de parámetros entrenables.

Esto hace que los modelos propuestos en dicha sección sean especialmente idóneos para entornos e infraestructuras *hardware* limitadas, debido a los reducidos tiempos de entrenamiento de estos respecto a los modelos planteados en la literatura por los autores de referencia.

Específicamente, el modelo que mejores resultados consigue es RoBERTa, obteniendo 0,866 tanto en exactitud como en F1, lo que supone una igualación de los resultados conseguidos por el mejor modelo propuesto en el artículo de referencia (que sería BERTweet) al conseguir estar en torno al 87%, pero en menos de la mitad de tiempo de entrenamiento, con 1985,500 segundos.

En caso de que ese tiempo de entrenamiento resultase ser un factor crítico, habría varias alternativas que escoger, dependiendo de cuánta exactitud y F1 se esté dispuesto a sacrificar a cambio de reducir este tiempo, pero en general se podría decir que la versión S de DeBERTaV3 sería el mejor modelo tras RoBERTa al ser capaz de obtener el segundo mejor resultado, de 0,853 en ambas métricas.

Capítulo 6

Conclusiones

Tal y como se describió en el Capítulo 2, el objetivo principal del presente trabajo era la obtención de modelos de IA que consiguiesen reconocer TEA a partir de mensajes en redes sociales, obteniendo en el proceso resultados que mejorasen los de la literatura científica.

Y, como se ha visto en el Capítulo 5, dicho objetivo se ha cumplido, puesto que se han logrado plantear modelos que han demostrado empíricamente ser competitivos o superiores a los propuestos en el artículo de partida [2].

A lo largo del planteamiento de distintos modelos pertenecientes a diversos paradigmas, ha quedado de manifiesto un patrón muy claro y repetido durante el desarrollo del trabajo: cuanto más complejo o sofisticado es el paradigma (partiendo del *Machine Learning* y acabando en modelos preentrenados de millones de parámetros), mejores resultados se obtienen, pero también mayor coste computacional hay que afrontar. Sin embargo, precisamente una de las aportaciones del presente trabajo es que es perfectamente posible la obtención de resultados que iguallen o mejoren a los de la literatura pero empleando modelos más eficientes, con menor tiempo de ejecución.

A pesar de esto, también hay que mencionar que este trabajo no está libre de limitaciones: que el conjunto de datos sea limitado y provenga de Twitter siempre será un factor limitante que haga que no se puedan extrapolar los patrones detectados a mensajes provenientes de otras redes sociales, y los modelos propuestos (especialmente los más

pesados) siempre se toparán con un techo de rendimiento al no contar con cantidades mayores de información.

De cara al futuro, sería posible plantear la obtención de mayores volúmenes de datos provenientes de fuentes más variadas, para así subsanar las limitaciones mencionadas: por un lado, intentar generalizar la detección de TEA a más espacios de interacción en línea, y, por otro, tratar de superar los techos de rendimiento al contar con más datos que permitan que los modelos extraigan información más sutil de los mismos.

Bibliografía

- [1] Autismo España. Detección y diagnóstico del autismo, 2025. Confederación Autismo España. Última modificación: abril de 2025. Accedido por última vez: junio de 2026. URL: <https://autismo.org.es/el-autismo/deteccion-y-diagnostico/>.
- [2] Sergio Rubio Martín, María Teresa García Ordás, Martín Bayón Gutiérrez, Natalia Prieto Fernández, and José Alberto Benítez Andrades. Enhancing ASD detection accuracy: a combined approach of machine learning and deep learning models with natural language processing. *Health Information Science and Systems*, 12(1):20, 2024. doi:10.1007/s13755-024-00281-y.
- [3] Eva Reviriego Rodrigo, Juan Carlos Bayón Yusta, Alicia Gutiérrez Iglesias, and Lucía Galnares-Cordero. Trastornos del espectro autista: evidencia científica sobre la detección, el diagnóstico y el tratamiento. Informe de evaluación de tecnologías sanitarias, Ministerio de Sanidad y Servicio de Evaluación de Tecnologías Sanitarias del País Vasco (OSTEBA), Vitoria-Gasteiz, 2022. Accedido por última vez: junio de 2026. URL: https://redets.sanidad.gob.es/documentos/OSTEBA_TEA.pdf.
- [4] Jaime Guillermo Carbonell, Ryszard Stanisław Michalski, and Tom Michael Mitchell. An overview of machine learning. In *Machine Learning*, pages 3–23. Morgan Kaufmann, San Francisco (CA), 1983. doi:10.1016/B978-0-08-051054-5.50005-4.
- [5] Ibomoiye Domor Mienye and Nobert Rangarirai Jere. A Survey of Decision Trees: Concepts, Algorithms, and Applications. *IEEE Access*, 12(1):86716–86727, 2024. doi:10.1109/ACCESS.2024.3416838.

- [6] Tianqi Chen and Carlos Guestrin. XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016(1):785–794, 2016. [doi:10.1145/2939672.2939785](https://doi.org/10.1145/2939672.2939785).
- [7] Thomas Cover and Peter Hart. Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1):21–27, 1967. [doi:10.1109/TIT.1967.1053964](https://doi.org/10.1109/TIT.1967.1053964).
- [8] Ranjan Kumar Mishra, Ganganapalli Yaddanapudi Sandesh Reddy, and Himanshu Pathak. The Understanding of Deep Learning: A Comprehensive Review. *Mathematical Problems in Engineering*, 2021(1):5548884, 2021. [doi:10.1155/2021/5548884](https://doi.org/10.1155/2021/5548884).
- [9] Seol-Hyun Noh. Analysis of Gradient Vanishing of RNNs and Performance Comparison. *Information*, 12(11):442, 2021. [doi:10.3390/info12110442](https://doi.org/10.3390/info12110442).
- [10] Alex Sherstinsky. Fundamentals of Recurrent Neural Network (RNN) and Long Short-Term Memory (LSTM) network. *Physica D: Nonlinear Phenomena*, 404:132306, 2020. [doi:10.1016/j.physd.2019.132306](https://doi.org/10.1016/j.physd.2019.132306).
- [11] Xing Ding and Jing Wang. The analysis of bidirectional long short-term memory network model for construction of cultural gene map and information extraction. *Scientific Reports*, 16(1):1395, 2025. [doi:10.1038/s41598-025-31611-z](https://doi.org/10.1038/s41598-025-31611-z).
- [12] Diksha Khurana, Aditya Koli, Kiran Khatter, and Sukhdev Singh. Natural language processing: state of the art, current trends and challenges. *Multimedia Tools and Applications*, 82(3):3713–3744, 2022. [doi:10.1007/s11042-022-13428-4](https://doi.org/10.1007/s11042-022-13428-4).
- [13] Kyle Aitken, Vinay V. Ramasesh, Yuan Cao, and Niru Maheswaranathan. Understanding how encoder-decoder architectures attend. In *Proceedings of the 35th International Conference on Neural Information Processing Systems*, pages 22184–22195. Curran Associates, 2021.

- [14] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186. Association for Computational Linguistics, 2019. [doi:10.18653/v1/N19-1423](https://doi.org/10.18653/v1/N19-1423).
- [15] Dat Quoc Nguyen, Thanh Vu, and Anh Tuan Nguyen. BERTweet: A pre-trained language model for English Tweets. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 9–14. Association for Computational Linguistics, 2020. [doi:10.18653/v1/2020.emnlp-demos.2](https://doi.org/10.18653/v1/2020.emnlp-demos.2).
- [16] Martin Frederick Porter. An algorithm for suffix stripping. *Program*, 14(3):211–218, 1980. [doi:10.1108/00330330610681286](https://doi.org/10.1108/00330330610681286).
- [17] Md Nafis Ul Alam, Kiran Basava, Ani Chitransh, H. M. Abdul Fattah, Hector D. Garcia-Verdugo, Shih-Hsuan Lo, Tanisha Lohchab, Kristen M. Martinet, Cristian Román-Palacios, Jhan Carlos Salazar, and Danielle Van Boxel. Machine learning in biological research: key algorithms, applications, and future directions. *BMC Biology*, 23(1):324, 2025. [doi:10.1186/s12915-025-02424-3](https://doi.org/10.1186/s12915-025-02424-3).
- [18] Jeffrey Pennington, Richard Socher, and Christopher Manning. Glove: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543. Association for Computational Linguistics, 2014. [doi:10.3115/v1/D14-1162](https://doi.org/10.3115/v1/D14-1162).
- [19] Stanford NLP Group. Glove: Global vectors for word representation, 2014. Stanford University. Última modificación: 2014. Accedido por última vez: junio de 2026. URL: <https://nlp.stanford.edu/projects/glove/>.
- [20] Hanhong Shi, Lei Wang, Rafał Scherer, Marcin Woźniak, Pengchao Zhang, and Wei Wei. Short-Term Load Forecasting Based on Adabelief Optimized Temporal

- Convolutional Network and Gated Recurrent Unit Hybrid Neural Network. *IEEE Access*, 9:66965–66981, 2021. [doi:10.1109/ACCESS.2021.3076313](https://doi.org/10.1109/ACCESS.2021.3076313).
- [21] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is All you Need. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, 2017.
- [22] James Lee-Thorp, Joshua Ainslie, Ilya Eckstein, and Santiago Ontañón. FNet: Mixing tokens with Fourier transforms. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4296–4313. Association for Computational Linguistics, 2022. [doi:10.18653/v1/2022.naacl-main.319](https://doi.org/10.18653/v1/2022.naacl-main.319).
- [23] Alan Victor Oppenheim and Ronald William Schaffer. *Discrete-Time Signal Processing*. Prentice Hall, 1999.
- [24] James William Cooley and John Wilder Tukey. An Algorithm for the Machine Calculation of Complex Fourier Series. *Mathematics of Computation*, 19(90):297–301, 1965. [doi:10.2307/2003354](https://doi.org/10.2307/2003354).
- [25] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. RoBERTa: A Robustly Optimized BERT Pretraining Approach. *Paul G. Allen School of Computer Science & Engineering*, 19(7):Article–01, 2019. [doi:10.48550/arXiv.1907.11692](https://doi.org/10.48550/arXiv.1907.11692).